Bocconi University
Master of Science in Artificial Intelligence

Master's Thesis

Learnable, Task-Adaptive Structured Memory for LLM Agents

Learning what to store *and* how to retrieve*: a parameter vector* $\boldsymbol{\theta}$*, optimized by black-box search, across grid worlds and six real long-context benchmarks.*

Candidate: Stefan Uifalean

Student ID: 3310360

Supervisor: Prof. Dirk Hovy

Academic Year 2025–2026

Declaration

I declare that this thesis is my own original work and that all sources and collaborations have been duly acknowledged. The code, data, and results supporting it are available in the public repository cited in Appendix A.

Stefan Uifalean
Milan, June 2026

Abstract

Memory in LLM-based agents is almost always *fixed*: a context window of a given size, a RAG index with a fixed chunking and retrieval rule, or a vector store that treats every event alike. Yet different tasks demand different memory structures — a multi-hop question needs entity relations, a navigation task needs spatial transitions, an end-of-corpus question needs recency-independent recall. This thesis asks whether an agent can **learn how to construct its own memory** — what to store, which concepts to track as nodes, and how to score retrieval — and whether that learned structure is **task-dependent**.

We parameterize memory construction by a vector $\boldsymbol{\theta}$ (storage probability, entity-creation threshold, temporal-edge probability, and learnable retrieval weights $w_{\text{graph}}/w_{\text{embed}}/w_{\text{recency}}$, up to ten dimensions in the most complete system, GraphMemoryV4) and optimize $\boldsymbol{\theta}$ — not the policy, not the value function — by black-box search (Evolution Strategy, then CMA-ES). The work proceeds in three stages of increasing realism. **Stage 1** (grid proof-of-concept) shows the optimizer recovers *different* $\boldsymbol{\theta}$ for different tasks and that adaptive $\boldsymbol{\theta}$ improves over the fixed-memory baseline (MultiHopKeyDoor success 2.5% $\rightarrow$ 27.5%). **Stage 2** scales to a twelve-memory-system $\times$ four-environment benchmark with ablation, zero-shot $\boldsymbol{\theta}$-transfer, and sensitivity analysis; the ten-dimensional GraphMemoryV4 reaches the top performance cluster, and $\boldsymbol{\theta}$ tuned for one long-haystack task transfers to another but not across task families. **Stage 3** moves to real LLMs (a `gpt-4o-mini` answerer) doing corpus-cumulative question answering over **six** real long-context benchmarks, with a corpus-mode CMA-ES re-tuning of $\boldsymbol{\theta}$ and a Claude cross-vendor judge.

The central Stage-3 finding, established after an adversarial self-audit, is that **corpus-cumulative tuning shifts $\boldsymbol{\theta}$ toward recency-independent, embedding-driven retrieval, producing an end-of-corpus accuracy lift that survives Holm-corrected held-out testing on two of the three domain-coherent benchmarks** (FinanceBench, CUAD; QASPER is non-significant), while two pre-registered domain-incoherent controls also lift and NarrativeQA is undefined by construction. The tuning objective is a validated proxy for answer quality (recall@8 vs. judge score, pooled Spearman $\rho = 0.69$). A full-context "dump-all" baseline is *statistically tied* with selective retrieval on accuracy but roughly 18$\times$ more expensive — so the case for learned, task-adaptive memory at this scale is **efficiency and scalability**, not an accuracy cliff.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Large language model (LLM) agents are increasingly deployed on tasks whose relevant information far exceeds a single prompt: answering a question that spans hundreds of pages of financial filings, recalling a fact stated five conversational sessions ago, or chaining evidence across several documents. The dominant response is to bolt on a *memory*: a context window of a fixed size, a retrieval-augmented generation (RAG) index with a fixed chunking and scoring rule, or a vector store that treats every observation alike.

What these solutions share is that the memory is **fixed**. The decision of *what to store*, *which concepts to track*, and *how to score a retrieval* is hand-designed once and frozen. Yet different tasks plainly demand different memory structures. A multi-hop question needs entity relations; a navigation task needs spatial and temporal transitions; an end-of-corpus question, asked after the agent has ingested an entire corpus, needs *recency-independent* recall, because the relevant fact may have been seen long ago. A memory rule that is optimal for one of these is often actively harmful for another.

A single parameter makes the tension concrete. Consider how much a retriever should favour *recent* evidence. A conversational assistant answering "what did I just ask you to change?" wants recency weighted heavily — the answer is almost always in the last few turns. An agent answering "what is the governing-law clause?" after ingesting a five-hundred-page contract wants recency weighted at essentially *zero* — the relevant clause was read hundreds of pages ago, and a recency-biased retriever will confidently surface the most recent (wrong) clause instead. The same is true of how aggressively to store (store everything, and retrieval drowns in noise; store too little, and the answer was never kept) and of how much to trust embedding similarity versus exact lexical overlap. No single setting of these knobs is good everywhere; the right setting is a property of the task. In practice this is "solved" by an engineer hand-tuning the memory for each deployment — ad hoc, unmeasured, and rarely revisited. The premise of this thesis is

that those knobs can instead be written down explicitly and *learned* per task, and that doing so both improves retrieval and makes the task-dependence measurable rather than anecdotal.

Why hand-frozen policies are the wrong default. The cost of freezing the memory policy is not merely that it is suboptimal; it is that the failure is *silent and mis-attributed*. When a store-everything vector index buries the governing-law clause under five hundred pages of near-duplicate boilerplate, or a store-too-little heuristic never retained the one sentence that answered the question, the downstream LLM does not report "my memory was mis-parameterized" — it hallucinates a fluent, wrong answer, and the engineer blames the prompt or the model. The store-everything and store-too-little failure modes are two ends of a single dilemma: aggressive storage maximizes recall but destroys precision at retrieval time, while conservative storage protects precision but discards evidence irrecoverably. A hand-frozen policy commits to one point on this trade-off before the task is known, and there is no point that is safe for every task. The recency knob discussed above is the sharpest instance — the same setting that a conversational assistant *must* have is the setting an end-of-corpus legal query *cannot* tolerate — but the same logic applies to storage aggressiveness and to the lexical-versus-semantic weighting. The premise of this thesis is that the right response is not a better fixed default but the removal of the fixed default: expose the trade-off as parameters and let the task select the operating point.

1.2 Research question

This thesis asks a single question:

> *Can an agent* ***learn how to construct its own memory*** *— what to store, which concepts to track as nodes, and how to score retrieval — and is the learned optimum* ***task-dependent****?*

We make the construction of memory itself a learnable object. A small parameter vector $\boldsymbol{\theta}$ governs storage, abstraction, and retrieval; we then *optimize* $\boldsymbol{\theta}$ — not the agent's policy and not the LLM's weights — with black-box search, and we ask whether the recovered $\boldsymbol{\theta}$ differs by task and whether adapting it improves end-task performance.

1.3 Contributions

The contribution is **not** a better reinforcement-learning policy or a new language model. It is the claim and the demonstration that *memory construction is a learnable, task-adaptive object*. Concretely, this thesis:

- **Formalizes memory construction as a parameter vector $\boldsymbol{\theta}$** (storage probability, entity/temporal thresholds, and learnable retrieval weights $w_{\text{graph}}/w_{\text{embed}}/w_{\text{recency}}$, up to ten dimensions in the most complete system, GraphMemoryV4) and optimizes it by Evolution Strategy and CMA-ES.
- **Shows task-dependence at three increasing levels of realism**: grid worlds (Stage 1), a twelve-system × four-environment benchmark with ablation, transfer, and sensitivity analysis (Stage 2), and corpus-cumulative question answering with real LLMs over six long-context benchmarks (Stage 3).
- **Subjects its own headline results to an adversarial self-audit** and reports the corrected numbers — including the finding that a full-context "dump-all" baseline, once a truncation bug was fixed, is statistically tied with selective retrieval on accuracy, which reframes the contribution of learned memory at this scale as one of *efficiency* rather than raw accuracy.

In keeping with that self-audit, the thesis is careful to separate four claims that a less disciplined account would blur into one. *First*, the accuracy of learned memory is **benchmark-dependent, not uniform**: against a same-budget tuned lexical baseline (Table 4.4) the learned V4_t memory wins decisively on FinanceBench, ties on QASPER, and *loses* on CUAD's legal clause-extraction — a result reported as such rather than averaged away, because the losing case pins down exactly where a single learned memory is the wrong tool. *Second*, the tuning is **near-deterministic**: at temperature zero the corpus-tuned optimum is stable across random seeds (Section 4.3), so the reported lifts are properties of the objective and not of a lucky search. *Third*, the task-optimal $\boldsymbol{\theta}$ is **predictable from cheap corpus descriptors**: a leave-one-benchmark-out predictor recovers most of the per-task tuning benefit (Figure 4.11, Section 4.4), which is what makes "learn a memory per task" operational rather than a per-deployment tuning chore. *Fourth*, and most importantly, the demonstrated advantage at long-context scale is one of **efficiency and scalability, not raw accuracy** — a distinction the thesis states plainly rather than letting an accuracy-competitive result masquerade as an accuracy win.

1.4 The central finding

Across the three stages the same claim holds: *what an agent stores and how it scores retrieval can be learned, and the learned optimum is task-dependent.* In the grids the optimizer recovers visibly different $\boldsymbol{\theta}$ per task and lifts the hardest task from 2.5% to 27.5% success. On the benchmark suite a ten-dimensional learnable memory reaches the top performance cluster, and a $\boldsymbol{\theta}$ tuned for one long-haystack task generalizes within that task family but not across families. On six real long-context benchmarks, re-tuning $\boldsymbol{\theta}$ on a corpus-cumulative objective yields a recency-independent memory whose

end-of-corpus accuracy lift **survives held-out testing on two of the three domain-coherent benchmarks** (FinanceBench and CUAD; QASPER is non-significant); two further benchmarks serve as pre-registered domain-incoherent controls, and NarrativeQA's objective is undefined by construction.

A second methodological result emerges from comparing learned selective memory against a fixed full-context baseline ("dump everything into the prompt"): once measured correctly, the two are *accuracy-competitive*. The argument for learnable, task-adaptive memory at this corpus scale is therefore **cost and scalability** — it matches accuracy at roughly one-eighteenth of the token spend and does not break when the corpus exceeds the context window — rather than a claim that selective retrieval is uniquely accurate.

The honest scope of the accuracy claim. It is worth stating precisely what is and is not established, because the temptation to overclaim is exactly what the self-audit was built to resist. The *learnable, task-dependent* claim is the robust one: across all three stages a single vector $\boldsymbol{\theta}$ governs what is stored and how retrieval is scored, and its recovered optimum differs by task in a way that is measured, not asserted. The *accuracy* claim is deliberately narrower. Learned memory is competitive rather than dominant: it wins where lexical structure is diffuse and recency is a trap, ties where a strong lexical baseline already suffices, and loses where the task is near-verbatim clause retrieval that a tuned lexical index does better (Table 4.4). Where the held-out lifts survive multiple-testing correction they are reported as surviving, and where they do not — QASPER — the null result is reported as a null result (Table 4.2). Two further limitations bound the claim rather than being hidden by it: the accuracy judge is a single LLM, so the reported agreement is self-consistency and not human adjudication, and the probe of judged accuracy against corpus size was inconclusive, so no claim of flat accuracy scaling is made. The scalability argument (Figure 4.10) rests instead on token cost and on the structural fact that selective memory does not break when the corpus outgrows the context window, which is a property of the mechanism and not of the judge.

1.5 Thesis outline

Chapter 2 reviews long-context retrieval, agent-memory systems, and black-box optimization, and positions this work. Chapter 3 formalizes $\boldsymbol{\theta}$, the graph-memory architecture, the optimizers, the three experimental settings, and the evaluation protocol. Chapter 4 reports the experiments stage by stage. Chapter 5 interprets the results, discusses threats to validity, and summarizes the adversarial self-audit. Chapter 6 concludes and lists future work. Four appendices give the reproducibility material, the adversarial self-audit, the complete per-benchmark tables, and the data-preparation and judging details.

Chapter 2

Background and Related Work

This chapter situates the thesis among three bodies of work: long-context retrieval for question answering, agent-memory systems for LLMs, and black-box optimization. It closes by stating precisely what is new here.

2.1 Long-context question answering and retrieval

When the evidence for a question exceeds the model's context window, the standard solution is retrieval-augmented generation (Lewis et al., 2020): an external index returns a small set of passages that are concatenated into the prompt. Retrieval quality is therefore decisive. Dense retrievers such as Dense Passage Retrieval (Karpukhin et al., 2020) and the unsupervised Contriever (Izacard et al., 2022) embed queries and passages into a shared space; a sparse lexical baseline, BM25 (Robertson and Zaragoza, 2009), remains strong when the query shares vocabulary with the target. More recent systems learn *when* and *what* to retrieve — Self-RAG (Asai et al., 2024) interleaves retrieval with self-critique, and LongRAG (Jiang et al., 2024) pairs long retrieval units with long-context readers — or impose hierarchy on the corpus, as in RAPTOR's recursive summary tree (Sarthi et al., 2024) and HiQA's hierarchical multi-document augmentation (Tao et al., 2024). In all of these the retrieval *policy* is either fixed or trained by gradient descent on in-domain supervision; none treats the storage-and-scoring rule as a small, task-adaptive parameter vector tuned by black-box search, which is the object of study here.

Two properties of this literature motivate the present work. First, retrieval quality is not one knob but several interacting decisions — what to index, how to chunk it, how to weight lexical versus semantic similarity, and how much to favour recent over old evidence — and the optimal setting of those decisions depends on the task: a multi-hop question over short encyclopaedic paragraphs rewards aggressive semantic matching, whereas a single-fact lookup in a long filing rewards precise lexical overlap, and a dialogue-memory task rewards recency. Systems that fix these decisions globally must therefore compromise

across tasks. Second, where the policy *is* learned, it is typically learned by gradient descent on large in-domain question–passage supervision, which both requires that supervision and entangles the retrieval rule with the parameters of a specific reader. The alternative pursued here — a handful of interpretable scalars optimized directly on a cheap retrieval objective, with no gradients and no reader in the loop — is comparatively under-explored, and it is precisely what makes *per-task* re-fitting practical.

2.2 Memory systems for LLM agents

A parallel line of work gives an agent a persistent, evolving memory rather than a one-shot retrieval index. MemGPT (Packer et al., 2023) organizes memory into tiers (core, archival, external) and lets the LLM page information in and out like an operating system. HippoRAG (Gutiérrez et al., 2024) builds a knowledge graph over the corpus and retrieves with Personalized PageRank. MemoryLLM (Wang et al., 2024) expands the model's hidden state with a self-updatable memory pool. Practitioner frameworks such as mem0 (Chhikara et al., 2024) and LangMem (LangChain, 2024) provide production-grade persistent memory, and StreamingLLM (Xiao et al., 2024) keeps an unbounded stream tractable with attention sinks. These systems are sophisticated, but their storage and retrieval *policies* are hand-designed; the decision of what to write to memory and how to weight a retrieval is not itself optimized, and certainly not re-optimized per task. This thesis fills exactly that gap.

It is worth being precise about *where* each system places its intelligence, because that locates the gap. MemGPT's intelligence is in a hand-written control loop that decides when to page between tiers; the paging heuristics are fixed by the designer. HippoRAG's is in an offline knowledge-graph construction step plus a Personalized-PageRank traversal whose behaviour is determined by the graph and a fixed damping factor, not tuned to the task. MemoryLLM moves the memory inside the model's hidden state, so updating the policy means retraining the model. Practitioner frameworks (mem0, LangMem) expose engineering knobs — retention windows, summary cadence — but leave their values to manual configuration. In every case the *construction policy* (what is stored, abstracted, and how candidates are scored at retrieval) is a fixed artifact of the system's design. The contribution here is orthogonal to all of them: it makes that policy a small optimized object, so the same memory architecture can be re-fit to a new task in minutes without touching the model or the control code. In principle the parameterization could be layered onto several of these systems, treating their retention and scoring heuristics as the free parameters.

Read/write asymmetry and the memory-tier taxonomy. It helps to separate two axes that the systems above conflate. The first is the *tier* structure — most designs

distinguish a small fast-access working context from a large slow archival store, with a promotion/eviction rule moving items between them; MemGPT's core/archival split (Packer et al., 2023) and StreamingLLM's sink-plus-window (Xiao et al., 2024) are two points on this spectrum, and the retrieval-augmented setting of Section 2.6 is the degenerate case of a single archival tier with no promotion. The second, and the one this thesis targets, is the *read/write asymmetry*: the write policy (what is committed to the store, at what granularity, and with what abstraction) is logically prior to and usually decoupled from the read policy (how a candidate is scored at query time), yet the two are jointly responsible for end-task recall. Committing more aggressively only pays off if the scorer can subsequently surface what was written, and a sharper scorer is wasted on a store that never recorded the relevant event. Existing systems fix each side independently and by hand; the single $\boldsymbol{\theta}$ here spans both, so the optimizer can trade a laxer write threshold against a sharper retrieval weight rather than tuning either in isolation. That the same architecture, re-fit per corpus, wins decisively on one benchmark yet loses on another (Table 4.4) is the empirical signature of this coupling: the best joint setting of write-and-read is genuinely corpus-specific, not a global constant that a hand-designed policy could have fixed once.

2.3 Black-box optimization of structured objects

Because the storage decision is discrete and non-differentiable (an event is either written or not), gradient descent is awkward and a black-box optimizer is natural. Evolution Strategies scale well to such settings (Salimans et al., 2017), and the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (Hansen, 2016) is a robust derivative-free optimizer for low-dimensional continuous vectors — exactly the regime of a ten-dimensional $\boldsymbol{\theta}$. Bayesian optimization (Snoek et al., 2012) is the main alternative for expensive black-box objectives and has been used to tune retrieval and embedding hyper-parameters. The novelty here is not the optimizer but the *object* it is pointed at: a memory-construction policy, tuned directly on a downstream recall objective.

CMA-ES is a deliberate choice for this regime rather than an arbitrary one. The objective is noisy (it depends on sampled questions and, through the stochastic write decision, on a random seed), non-separable (the storage thresholds and the retrieval weights interact — storing more only helps if the scorer can then surface it), and, as the Stage-2 sensitivity analysis shows, anisotropic and sharply ridged. CMA-ES is built for exactly these conditions: it maintains a full covariance matrix over the search distribution and adapts it each generation, so it learns the local correlation structure of the landscape and takes large steps along flat directions and small steps across sharp ones — behaviour a coordinate-wise or fixed-step search cannot match. Bayesian optimization is the natural competitor for very expensive objectives where each evaluation must be hoarded, but

its surrogate-model overhead buys little at ten dimensions with cheap (LLM-free) recall evaluations, and it copes less gracefully with the objective's noise. Gradient methods are inapplicable because the write decision is discrete. We therefore use a simpler Evolution Strategy for the small Stage-1 search and CMA-ES for Stages 2–3; the recurring empirical observation that the optimizer drives several parameters to a boundary or to zero (notably the graph-traversal weight) is itself diagnostic of which components carry load.

Why a full-covariance ES, and why not the cheaper alternatives. The deeper reason to prefer CMA-ES over the simpler evolution strategies of Salimans et al. (2017) is that those methods estimate only a diagonal or isotropic search distribution: they rescale each coordinate independently but cannot rotate the sampling ellipse to align with a diagonal ridge. When the storage thresholds and the retrieval weights trade off against one another, the profitable search directions are linear combinations of parameters, not single axes, and a diagonal method must zig-zag across the ridge instead of travelling along it. CMA-ES's rank-μ and rank-one covariance updates recover exactly that rotation from the successful samples of each generation, which is why it is the standard choice for low-dimensional, non-separable, noisy objectives (Hansen, 2016). The noise deserves emphasis: because the write decision is sampled, a single $\boldsymbol{\theta}$ does not map to a single score but to a distribution over scores, so the optimizer is really performing stochastic optimization, and the population-based estimate of the gradient of the expected objective is far more robust to this than a surrogate fit to a handful of noisy point evaluations. That the objective turns out to be near-deterministic at $T{=}0$ (Section 2.6) is a convenient *a posteriori* property, not something the choice of optimizer relied on; an optimizer that only worked on quiet objectives would have been the wrong tool for a discrete-write policy. Bayesian optimization (Snoek et al., 2012) inverts the cost trade-off: it is designed to hoard information when each evaluation is enormously expensive, spending model-fitting compute to choose the next point wisely, but at ten cheap LLM-free recall evaluations the surrogate's overhead and its own noise assumptions buy little over simply sampling a population.

2.4 Structured and graph memory

Representing memory as a graph of typed nodes (events, entities) and edges (temporal, mention) is a recurring idea, from knowledge-graph RAG (Gutiérrez et al., 2024) to entity-centric reading comprehension. Our graph-memory implementation uses NetworkX (Hagberg et al., 2008) and adds a learnable retrieval score that linearly combines a graph-traversal term, an embedding-similarity term, and a recency term, with weights that are part of $\boldsymbol{\theta}$. A separate exploratory strand, verbal self-reflection (Reflexion, Shinn et al., 2023), is discussed as a negative result in Appendix C.

Knowledge-graph retrieval and what a learnable scorer generalizes. Graph-memory retrieval typically proceeds in two stages: an offline construction pass links passages to entities and lays down typed edges, and an online traversal ranks nodes by their graph-structural relevance to the query's seed entities. Personalized PageRank is the canonical traversal — it diffuses probability mass from the seeds through the edge structure and reads relevance off the stationary distribution (Gutiérrez et al., 2024) — but its behaviour is fixed by the graph topology and a single damping constant that trades multi-hop reach against locality, and that constant is not tuned to the task. Our formulation subsumes this as one term among several: the learnable score combines a graph-traversal component, an embedding-similarity component, and a recency component in a single linear rule whose weights live in $\boldsymbol{\theta}$. Setting the embedding and recency weights to zero recovers a pure structural ranker; letting the optimizer move them lets a corpus decide how much of its relevance signal is topological rather than semantic or temporal. The informative outcome is that the graph-traversal weight is repeatedly driven toward zero, which we read not as evidence that structure is useless but as evidence that on these corpora the entity-graph adds little *beyond* what dense similarity already captures — a scaffold rather than a load-bearing retrieval channel. That negative signal is only legible because the traversal weight is an exposed, optimized parameter rather than a hard-wired damping constant; a fixed-policy graph retriever cannot report that its own structure was redundant.

2.5 Benchmarks for long-context question answering

Stage 3 evaluates on six published benchmarks chosen to span disjoint domains: HotpotQA for multi-hop Wikipedia reasoning (Yang et al., 2018), QASPER for information-seeking questions over NLP papers (Dasigi et al., 2021), CUAD for expert-annotated legal-contract review (Hendrycks et al., 2021), NarrativeQA for abstractive questions over full books (Kočiský et al., 2018), FinanceBench for questions over SEC filings (Islam et al., 2023), and LongMemEval for long-term interactive dialogue memory (Wu et al., 2025). Answers are generated by `gpt-4o-mini` and scored by a Claude judge (cross-vendor relative to the answerer), following the LLM-as-judge methodology and its known biases (Zheng et al., 2023); embeddings use the distilled MiniLM sentence encoder (Wang et al., 2020; Reimers and Gurevych, 2019).

The six are not interchangeable, and the spread is the point. They differ in document length (a HotpotQA paragraph versus a full NarrativeQA book), in question style (lexical-overlap fact lookup in FinanceBench versus paraphrased information-seeking in QASPER), and in what "memory" even means (a single filing versus a multi-session dialogue history in LongMemEval). If the optimal $\boldsymbol{\theta}$ were the same across such different corpora, the central task-dependence claim would be false; the diversity is therefore a

Table 2.1: Positioning against representative prior memory / retrieval systems. "Task-adaptive" = the policy is re-fit per task rather than fixed; "Parametric policy" = the storage/retrieval behavior is governed by exposed numeric parameters that can be optimized.

System	What it optimizes	Task-adaptive?	Parametric po
Dense / BM25 RAG	retrieval ranking only	no	no (fixed sco
MemGPT (Packer et al., 2023)	paging policy (hand-designed)	no	no
HippoRAG (Gutiérrez et al., 2024)	graph-indexed retrieval	no	no
Reflexion (Shinn et al., 2023)	verbal self-feedback	per episode	no
This thesis	storage *and* abstraction *and* retrieval scoring (one $\boldsymbol{\theta}$)	**yes** (CMA-ES per task)	**yes** (10-D

test, not decoration. Three of the six — FinanceBench, CUAD, QASPER — are *domain-coherent*: their questions target persistent facts that end-of-corpus retrieval should recover, so the hypothesis predicts a lift. HotpotQA and LongMemEval serve as pre-registered *domain-incoherent controls* (their distractor / session structure means corpus-cumulative recall is not the natural objective), and NarrativeQA is included despite providing no paragraph-level gold, which makes its recall objective undefined by construction. This coherent/control/undefined partition, fixed before judging, is what lets the Stage-3 results be read as a test rather than a search for a favourable subset.

2.6 Positioning: what is new

Relative to the work above, this thesis contributes (i) a single parameter vector $\boldsymbol{\theta}$ that governs the *entire* memory-construction pipeline — storage, abstraction, and retrieval scoring — rather than any one stage; (ii) *per-task* optimization of that vector by black-box search, with the transfer behavior (within- vs. across-family) measured rather than assumed; and (iii) a corpus-cumulative evaluation on six real benchmarks with gold relevance signals, cross-vendor LLM judging, held-out significance testing, and a disclosed adversarial self-audit. To our knowledge, no prior system tunes a memory-construction policy per task with a derivative-free optimizer and reports its transfer and its comparison against a full-context baseline.

Table 2.1 places the contribution against the most relevant prior systems along four axes: what the system optimizes, whether that adaptation is *per task*, whether the memory policy is exposed as tunable parameters (as opposed to a fixed hand-designed pipeline), and the evaluation scope.

Chapter 3

Methodology

3.1 Problem formulation

We treat memory construction as a function parameterized by a vector $\boldsymbol{\theta}$. As the agent observes a stream of events, $\boldsymbol{\theta}$ governs three decisions: *storage* (is this event written to memory?), *abstraction* (does it create or update an entity node, a temporal edge?), and *retrieval scoring* (how is a stored item ranked against a query?). Crucially, we do *not* learn the agent's policy or the language model's weights; we learn only $\boldsymbol{\theta}$, holding everything else fixed. This isolates the question of interest — is the optimal memory-construction rule task-dependent? — from confounds in policy learning.

Let $R(\boldsymbol{\theta}; \mathcal{T})$ be the agent's mean performance on task $\mathcal{T}$ when its memory is built with $\boldsymbol{\theta}$. We seek $\boldsymbol{\theta}^\star_{\mathcal{T}} = \arg\max_{\boldsymbol{\theta}} R(\boldsymbol{\theta}; \mathcal{T})$ for each task, and ask (i) whether $\boldsymbol{\theta}^\star_{\mathcal{T}}$ visibly differs across tasks, and (ii) whether $\boldsymbol{\theta}^\star_{\mathcal{T}}$ transfers to a related task $\mathcal{T}'$.

Two design choices make this formulation worth stating precisely. First, fixing the policy and the language model is what gives the experiment its *attributive* power: any change in R is caused by the memory-construction rule and nothing else, so a difference in $\boldsymbol{\theta}^\star$ across tasks cannot be explained away as the policy adapting or the model being fine-tuned. The cost is that we measure the ceiling a *learned memory* can reach under a fixed reader, not the ceiling of an end-to-end system — an honest scoping we keep throughout. Second, the formulation is deliberately *falsifiable*: if a single $\boldsymbol{\theta}^\star$ were optimal for every task, question (i) would return "no different" and the thesis's central claim would be refuted; if $\boldsymbol{\theta}^\star_{\mathcal{T}}$ always transferred, question (ii) would make per-task tuning unnecessary. Neither outcome is assumed. We report $\boldsymbol{\theta}^\star$ vectors side by side so a reader can see the task-dependence directly (Stage 1), measure transfer rather than assert it (Stage 2), and — in the extension of this work — ask the sharper question of whether $\boldsymbol{\theta}^\star$ is *predictable* from corpus statistics even when it is not constant (Section 4.4).

Table 3.1: Notation. The first ten rows are the components of $\boldsymbol{\theta}$ (jointly optimized); the remainder are evaluation symbols.

Symbol	Meaning
θ_{store}	base storage propensity (bias on the write decision)
θ_{novel}	weight on novelty (distance from already-stored events) in the write decision
θ_{surprise}	weight on prediction-surprise in the write decision
θ_{erich}	weight on the learned importance/enrichment score
θ_{entity}	entity-node creation threshold
θ_{temporal}	temporal-edge creation probability
θ_{decay}	exponential decay rate applied to node weights over time
w_{graph}	retrieval weight on the graph-traversal signal g
w_{embed}	retrieval weight on embedding cosine similarity
w_{recency}	retrieval weight on the recency term ρ
$\boldsymbol{\theta}$	the full 10-D parameter vector (all rows above)
$s(e, q)$	retrieval score of event e for query q (Eq. 3.1)
recall@k	fraction of gold evidence retrieved in the top k (CMA-ES objective)
V4_{t}	the corpus-mode text variant of GraphMemoryV4 used in Stage 3

3.2 The parameter vector and the V1→V4 progression

The memory is a graph (Section 3.3). The earliest version (V1) exposes three parameters: θ_{store} (storage probability), θ_{entity} (entity-creation threshold), and θ_{temporal} (temporal-edge probability); the default $(1, 0, 1)$ reproduces a fixed store-everything baseline. The most complete system, GraphMemoryV4, extends this to ten dimensions by adding a learned importance score and three learnable retrieval weights. An item's retrieval score against a query is

$$s(\text{item}, q) \;=\; w_{\text{graph}}\, g(\text{item}, q) \;+\; w_{\text{embed}}\, \cos(e_{\text{item}}, e_q) \;+\; w_{\text{recency}}\, \rho(\text{item}), \tag{3.1}$$

where g is a graph-traversal signal, $\cos(\cdot)$ is embedding similarity, and ρ is a recency term; the storage/abstraction parameters $(\theta_{\text{store}}, \theta_{\text{novel}}, \theta_{\text{entity}}, \theta_{\text{temporal}}, \theta_{\text{surprise}}, \theta_{\text{erich}}, \theta_{\text{decay}})$ gate what enters the graph and how it decays. All ten components live in $\boldsymbol{\theta}$ and are optimized jointly. The MiniLM-era GraphMemoryV4 optimum used in Stages 2–3 is $\boldsymbol{\theta} \approx$ (θ_{store}=0.35, θ_{novel}=0.44, θ_{temporal}=0.73, θ_{decay}=0.72, w_{graph}=1.19, w_{embed}=1.31, w_{recency}=1.21) (remaining components omitted for brevity). Table 3.1 summarizes the full notation used throughout the thesis.

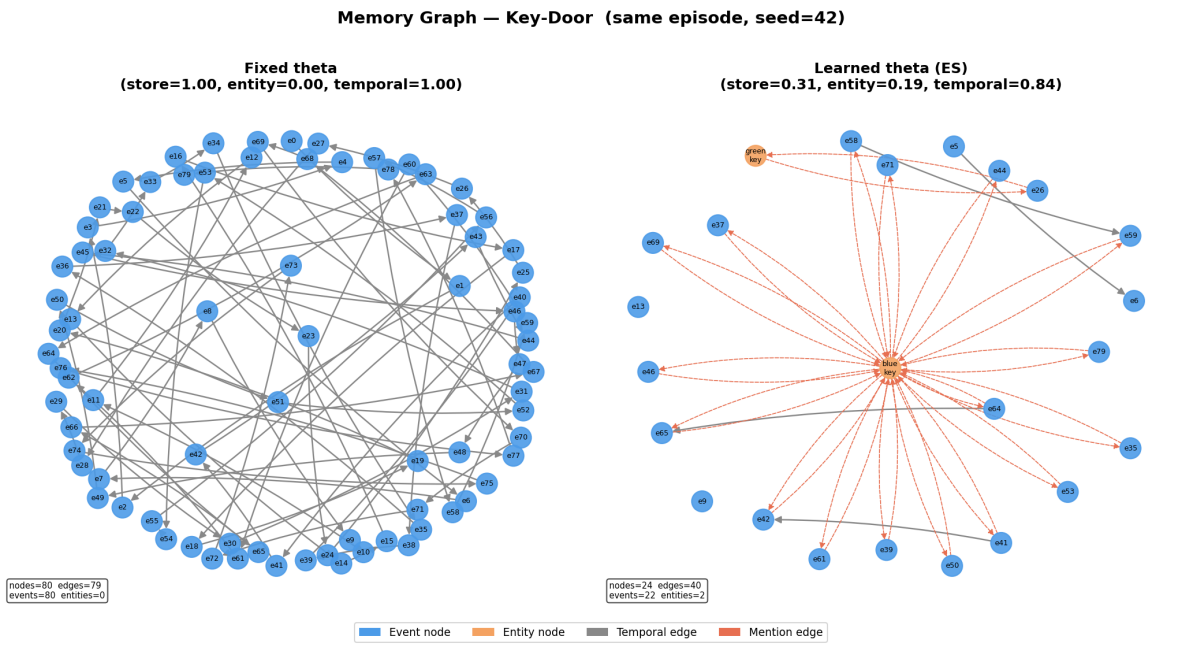


Figure 3.1: Memory graph for one Key-Door episode under the fixed baseline $\boldsymbol{\theta} = (1, 0, 1)$ (left, dense) versus a learned $\boldsymbol{\theta}$ (right, sparse and entity-centric). The learned rule stores far fewer events but keeps temporal structure.

3.3 Graph-memory architecture

Memory is a directed graph built with NetworkX (Hagberg et al., 2008). *Event* nodes hold observations; *entity* nodes are created when an observation's novelty exceeds θ_{novel}; *temporal* edges link consecutive retained events and *mention* edges link events to the entities they reference. Retrieval embeds the query, scores every node by Equation (3.1), and returns the top k (we use $k = 8$ throughout unless stated). In the Stage-3 text variant V4_{t} the Bayesian entity gate is bypassed so that document text is treated uniformly; all other $\boldsymbol{\theta}$ components are unchanged. Figure 3.1 illustrates how a learned $\boldsymbol{\theta}$ produces a sparse, entity-centric graph where the fixed baseline produces a dense, undifferentiated one.

3.4 Optimization

Because storage is discrete and the objective is non-differentiable, we optimize $\boldsymbol{\theta}$ with a derivative-free search. CMA-ES (Hansen, 2016) is well-suited to this regime: the search space is low-dimensional (ten continuous parameters), the objective is noisy (stochastic episodes and sampled questions) and non-differentiable (storage is a discrete write decision), and each evaluation is moderately expensive. Gradient methods do not apply; Bayesian optimization (Snoek et al., 2012) offers little advantage at this dimensionality, whereas CMA-ES's covariance adaptation copes with the anisotropic, sharply-peaked landscape

we observe empirically in the sensitivity analysis (Section 4.2). Stage 1 uses a simpler Evolution Strategy (12 generations × 6 candidates × 40 episodes). Stages 2–3 use CMA-ES (Hansen, 2016) (30 generations × 50 episodes per candidate for the benchmark; a corpus-mode variant for Stage 3). In Stage 3 the CMA-ES objective is pure recall@k of the gold evidence — *no LLM is in the optimization loop* — so tuning is inexpensive and the LLM is used only at evaluation time.

3.4.1 Same-budget baseline tuning

A learned memory that beats an *untuned* retriever proves little: any configuration handed a search budget will outrun one handed none. The honest counterfactual is therefore not a fixed default but a classical retriever that is allowed to spend the *same* corpus-mode tuning budget as $\boldsymbol{\theta}$. We grant each baseline the identical resource $\boldsymbol{\theta}$ receives — the same corpus-cumulative ingestion, the same tuning-question split, and the same recall@k objective (Algorithm 1) — and search its free parameters on exactly that objective. Two tuners are run per benchmark. The first is a grid search over the Okapi BM25 term-saturation and length-normalization coefficients (k_1, b), scored by corpus recall@k of the gold evidence, so the lexical baseline is placed at *its own* corpus-fitted optimum rather than at library defaults. The second is a sweep of the attention-memory temperature τ, which controls how sharply the softmax over candidate items concentrates mass; low τ approximates hard top-k selection and high τ diffuses it, so the sweep locates the baseline's best precision/recall trade-off under the shared budget. Only after each baseline sits at its tuned optimum do we compare it to the tuned $\boldsymbol{\theta}$ (Table 4.4). This construction converts the headline into a *same-budget* contrast: whatever separation survives cannot be attributed to one side having been tuned and the other left at defaults, and a benchmark on which the tuned lexical baseline *wins* is then genuine evidence about where learned memory does not help, not an artifact of an unfair baseline.

Algorithm 1 states the corpus-cumulative tuning loop and Algorithm 2 the per-event memory update that each candidate $\boldsymbol{\theta}$ induces.

3.5 The three experimental settings

Stage 1 — grid worlds. Four grid environments (Key-Door, Goal-Room, MultiHop-KeyDoor, MegaQuestRoom) of increasing difficulty. The agent reads hint signs, must remember which key opens which door, and is rewarded for reaching the goal. These are deliberately simple so that the learned $\boldsymbol{\theta}$ can be inspected directly.

Stage 2 — benchmark suite. Twelve memory systems (including GraphMemoryV4, a neural-controller variant, and standard baselines such as working-memory, episodic-

Algorithm 1 Corpus-cumulative CMA-ES tuning of $\boldsymbol{\theta}$ (Stage 3)

Require: corpus $\mathcal{D} = (d_1, \ldots, d_T)$, tuning questions $\mathcal{Q}_{\text{tune}}$, generations G, population λ

initialize CMA-ES mean $\boldsymbol{m} \leftarrow \boldsymbol{\theta}_0$, step size σ, covariance $C \leftarrow I$
for $g = 1 \ldots G$ **do**
  for $i = 1 \ldots \lambda$ **do** ▷ evaluate one candidate
    sample $\boldsymbol{\theta}_i \sim \mathcal{N}(\boldsymbol{m}, \sigma^2 C)$
    $M \leftarrow \emptyset$ ▷ empty memory
    for $t = 1 \ldots T$ **do** ▷ ingest corpus cumulatively
      $M \leftarrow \textsc{MemoryUpdate}(M, d_t, \boldsymbol{\theta}_i)$ ▷ Algorithm 2
    end for
    $f_i \leftarrow \frac{1}{|\mathcal{Q}_{\text{tune}}|} \sum_{q \in \mathcal{Q}_{\text{tune}}} \textsc{Recall@}k\Big(\textsc{Retrieve}(M, q, \boldsymbol{\theta}_i),\ \text{gold}(q)\Big)$
  end for
  update $\boldsymbol{m}, \sigma, C$ from the μ best $\{(\boldsymbol{\theta}_i, f_i)\}$ ▷ standard CMA-ES recombination
end for
return $\boldsymbol{\theta}^\star = \arg\max_i f_i$ over all generations

Algorithm 2 MemoryUpdate: per-event store / abstract / score, governed by $\boldsymbol{\theta}$

Require: memory M, incoming document d, parameters $\boldsymbol{\theta}$

for each event e extracted from d **do**
  $\text{nov}(e) \leftarrow 1 - \max_{e' \in M} \cos(\text{emb}(e), \text{emb}(e'))$ ▷ novelty vs. stored
  if $\theta_{\text{store}} + \theta_{\text{novel}} \cdot \text{nov}(e) + \theta_{\text{surprise}} \cdot \text{surp}(e) > 0$ **then**
    add e to M with timestamp; link typed edges (entity / temporal) into the graph
    apply decay $w \leftarrow w \cdot e^{-\theta_{\text{decay}}\, \Delta t}$ to existing nodes
  end if
end for
at retrieval for query q, score each $e \in M$:
  $s(e, q) = w_{\text{graph}} \cdot \text{graph}(e, q) + w_{\text{embed}} \cdot \cos(\text{emb}(e), \text{emb}(q)) + w_{\text{recency}} \cdot \text{recency}(e)$
return top-k events by $s(\cdot, q)$

semantic, summary, attention, and RAG memories) are evaluated on the four environments, with ablation of each $\boldsymbol{\theta}$ component, zero-shot $\boldsymbol{\theta}$-transfer across environments, and a 2-D sensitivity sweep.

Stage 3 — corpus-cumulative LLM QA. Six published long-context benchmarks (Section 2.6) are ingested document-by-document into the V4_t memory. Questions are answered in two regimes: *online* (immediately after the source document is ingested, so it is recent) and *batch* (at the end of the corpus, against the full accumulated memory). $\boldsymbol{\theta}$ is re-tuned per benchmark on the corpus-cumulative recall@k objective.

3.6 Evaluation protocol

Two metrics are used. *Retrieval* quality is measured by recall@k of the gold evidence paragraphs, which requires no LLM and drives CMA-ES. *Answer* quality is measured by an LLM-as-judge protocol: a `gpt-4o-mini` answerer produces an answer from the k retrieved items, and a **Claude** judge — cross-vendor relative to the answerer, to avoid the self-preference bias a same-model judge would introduce (Zheng et al., 2023) — scores it on a five-point rubric $\{0, 0.25, 0.5, 0.75, 1\}$. Judging is two-tier: content answers are scored one-by-one by the Claude judge, while refusals and "I don't know" acknowledgments are scored by a validated rule-assisted classifier (precision/ recall reported in Appendix B; population-weighted score error 0.028 on a 300-entry hand-checked sample). Judge reliability is quantified by an independent blind second pass over a stratified 180-question sample (Appendix B): quadratic-weighted Cohen's $\kappa = 0.66$ (*substantial* agreement; 87.8% of scores agree within one rubric level). Because both passes are Claude-class, this measures judge *self-consistency* rather than cross-vendor or human agreement. All runs are seeded, and every result carries a provenance record (commit identifier, embedding backend, and dataset fingerprint) that is automatically checked for consistency between the evaluation inputs and the reported outputs.

3.6.1 Two-tier judging and the rule-assisted refusal path

The two-tier design exists because a single scoring path would misprice the two kinds of output an answerer produces. A substantive answer requires reading the gold evidence and judging semantic adequacy — an irreducibly LLM-shaped task, so those entries are scored one-by-one by the cross-vendor Claude judge. A refusal or an "I don't know" acknowledgment, by contrast, carries almost no content to weigh: the entire scoring decision is whether the abstention was warranted given that the evidence was (or was not) retrievable. Routing such entries through a generative judge would spend tokens on a near-deterministic call and, worse, expose an honest abstention to the judge's own answer preferences. We instead classify them with a rule-assisted refusal classifier and assign the score its label inherits.

Because a share of every benchmark's scores flows through this cheaper path, the classifier is validated as a measurement instrument in its own right, not assumed correct. We hand-check a stratified sample of entries and report the classifier's precision and recall against those labels (Appendix B). The decisive quantity, however, is not classification accuracy but its *effect on the reported metric*: because a mislabeled entry changes a benchmark's mean only by the score difference it induces, we propagate every classifier error into the aggregate and report the resulting per-benchmark mean-score perturbation, together with the pooled per-entry error rate. The rule-assisted share varies sharply by benchmark — it is substantial on the corpora where terse "not stated" abstentions

are common and near-zero on narrative corpora where answers are prose — which is precisely why a single global error figure would be misleading and the per-benchmark breakdown (Appendix B) is the honest reporting unit. Only after confirming that the induced mean-score error is small on every benchmark do we treat the two paths as a single comparable metric in the headline tables.

Significance and held-out testing. Because the headline comparison is a learned configuration against its own untuned baseline, two safeguards guard against over-fitting and small-sample noise. First, every reported lift is recomputed on a *held-out* question split that the CMA-ES tuner never saw: the corpus is partitioned by a fixed, documented rule before tuning, $\boldsymbol{\theta}$ is optimized on the tuning split alone, and the lift is measured on the held-out split — so a $\boldsymbol{\theta}$ that merely memorized the tuning questions would show no held-out gain. Second, significance is assessed with non-parametric, paired tests appropriate to bounded judge scores: a Wilcoxon signed-rank test on the per-question (tuned − canonical) differences, with Holm–Bonferroni correction across the benchmark family so that reporting six benchmarks does not inflate the false-positive rate. Effect sizes are reported as bootstrap (or, where questions cluster within documents, cluster-bootstrap) 95% confidence intervals rather than point estimates alone. A lift is called "significant" only when it survives the held-out split *and* the Holm-corrected test; everything weaker is reported as directional or non-significant, never rounded up.

Chapter 4

Experiments and Results

The three stages answer the research question at increasing realism. Stage 1 shows the optimum is task-dependent; Stage 2 shows a learnable memory is competitive and characterizes what each parameter does; Stage 3 tests the idea on real LLMs over six long-context corpora.

4.1 Stage 1 — task-dependent $\boldsymbol{\theta}$ in grid worlds

An Evolution Strategy optimizes the three-dimensional $\boldsymbol{\theta}$ separately on each grid environment. Table 4.1 reports the recovered vectors and the success rate against the fixed-memory baseline.

The headline is the *difference* between the recovered vectors: Key-Door discards most events (θ_{store}=0.12) but preserves temporal order; Goal-Room stores everything. This is the first, cleanest evidence that the optimal memory structure is task-dependent (Figure 4.1). On the hardest task the learned rule lifts success from 2.5% to 27.5%; we report this as an absolute lift of $\sim$25 points (the baseline is near-zero, so a percentage-ratio framing would overstate it), on a single-seed proof of concept.

4.2 Stage 2 — benchmark, ablation, transfer, sensitivity

Top-cluster performance. On the hardest grid task (MultiHopKeyDoor), the CMA-ES-tuned ten-dimensional GraphMemoryV4 reaches the top performance cluster (Figure 4.2): mean reward 0.178 versus the prior best EpisodicSemantic at 0.173. We state this as a *statistical tie*, not a clean "#1": it is a single-seed point estimate and EpisodicSemantic's 95% bootstrap confidence interval $[0.120, 0.220]$ contains 0.178. The defensible claim is that the ten-dimensional parameterization *reaches the top cluster*, a +75% improvement over its own untuned starting point, not that it is uniquely best.

Table 4.1: Stage 1: learned $\boldsymbol{\theta} = (\theta_{\text{store}}, \theta_{\text{entity}}, \theta_{\text{temporal}})$ per environment, and baseline vs. learned success. The vectors are visibly different per task.

Environment	Learned $\boldsymbol{\theta}$	Baseline	Learned
Key-Door	(0.116, 0.000, 0.819)	17.5%	30.0%
Goal-Room	(1.000, 0.220, 1.000)	70.0%	80.0%
MultiHopKeyDoor	(1.000, 0.487, 0.843)	**2.5%**	**27.5%**

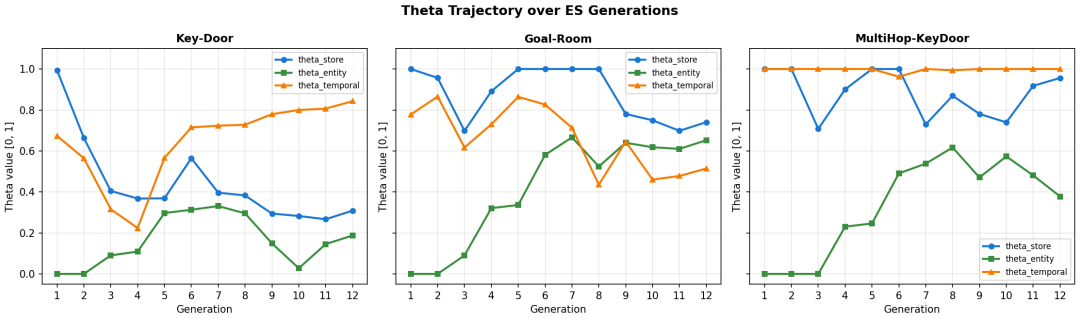


Figure 4.1: Stage 1: $\boldsymbol{\theta}$ trajectories over Evolution-Strategy generations for three environments. The optimizer converges to visibly different vectors, which is the central Stage-1 claim.

Ablation. Removing each $\boldsymbol{\theta}$ component one at a time (Figure 4.3a) shows that θ_{novel} is the dominant, load-bearing dimension: zeroing it collapses reward to 0 (it gates the entire storage pipeline). Other components carry less, and a w_{graph} sweep (Figure 4.3b) shows the graph-traversal term carries *no* measurable load on this task: zeroing it barely changes reward. The corpus-tuning signature is therefore a three-shift rather than a four-shift; the graph term behaves as a typed storage scaffold rather than a relational retrieval engine.

Transfer and sensitivity. A $\boldsymbol{\theta}$ tuned on one environment transfers strongly to a structurally similar one (Goal-Room 0.69, HardKeyDoor 0.17) but fails completely on a larger out-of-distribution task (MegaQuestRoom 0.00), even though its retrieval there is high-precision — the failure is task scale, not memory quality (Figure 4.4a). The 2-D sensitivity landscape (Figure 4.4b) shows a *sharp peak*: reward collapses once θ_{novel} drops below $\sim$0.3, so the optimizer must find a fairly precise value. A neural-controller variant (a $\sim$2,000-parameter MLP tuned by the same CMA-ES budget) matches but does not clearly exceed the ten-scalar GraphMemoryV4; details are in Appendix C. The Pareto view (Figure 4.5) confirms GraphMemoryV4 is on the reward-vs-cost frontier.

4.3 Stage 3 — six real long-context benchmarks

Each benchmark is ingested document-by-document into the V4_{t} memory; questions are answered *online* and *batch* and scored by the cross-vendor Claude judge (Section 3.6). $\boldsymbol{\theta}$ is re-tuned per benchmark on the corpus-cumulative recall@k objective.

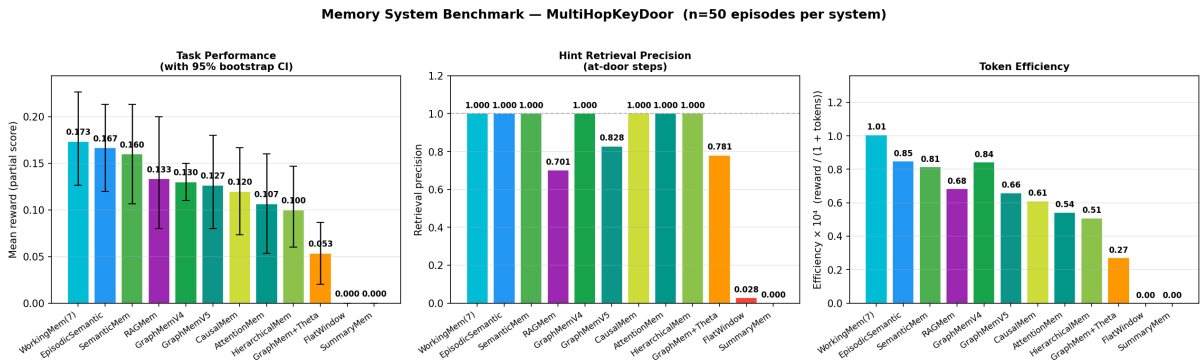
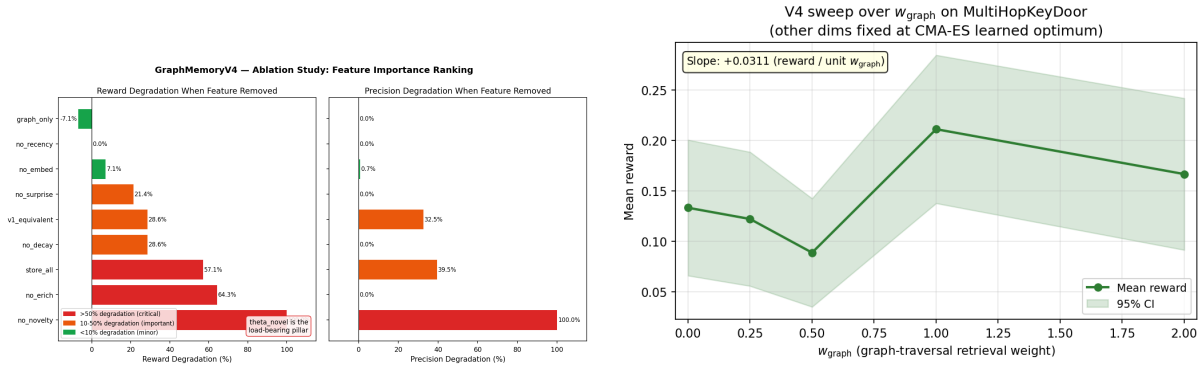


Figure 4.2: Stage 2: twelve memory systems on MultiHopKeyDoor (task performance with 95% bootstrap CIs, retrieval precision, token efficiency). GraphMemoryV4 sits in the top cluster, statistically tied with the strongest baselines.



(a) Per-component degradation ranking.

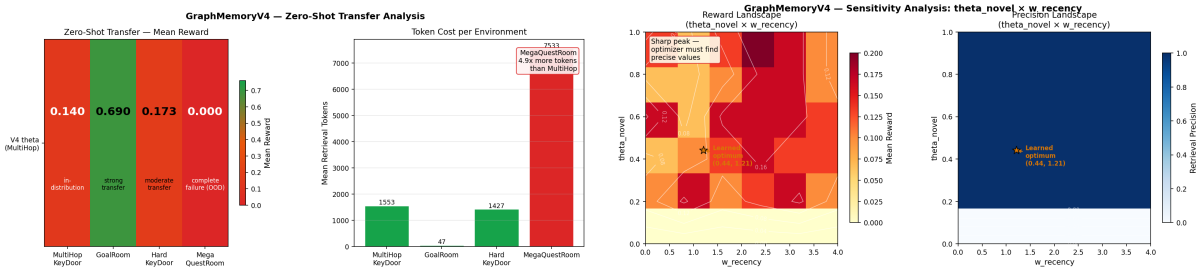
(b) w_{graph} sweep: flat (no load).

Figure 4.3: Stage 2 ablation. θ_{novel} is load-bearing; the graph term carries no measurable retrieval load.

Headline: end-of-corpus accuracy lift. Table 4.2 reports the batch (end-of-corpus) judge means under the canonical (grid-tuned) $\boldsymbol{\theta}$ and the corpus-tuned $\boldsymbol{\theta}$, with the held-out significance verdict.

Of the three *domain-coherent* benchmarks (where the hypothesis predicts a lift), two — FinanceBench and CUAD — show a held-out lift that survives Holm correction; QASPER does not. HotpotQA and LongMemEval are pre-registered *domain-incoherent controls* (no lift predicted), and NarrativeQA's tuning objective is undefined by construction. We therefore lead with the count that the design supports: **two of three coherent benchmarks show a significant end-of-corpus lift**.

The mechanism is consistent across benchmarks: corpus tuning drives w_{recency} toward 0 and w_{embed} up, producing a memory that does *not* depend on the queried document being recent — exactly what end-of-corpus QA needs. In *online* mode (fresh document) the recency-heavy canonical $\boldsymbol{\theta}$ is competitive or better, so the lift is genuinely a batch-mode effect (Figure 4.6).



(a) Zero-shot $\boldsymbol{\theta}$-transfer. **(b)** Sensitivity landscape (sharp peak).

Figure 4.4: Stage 2 transfer and sensitivity.

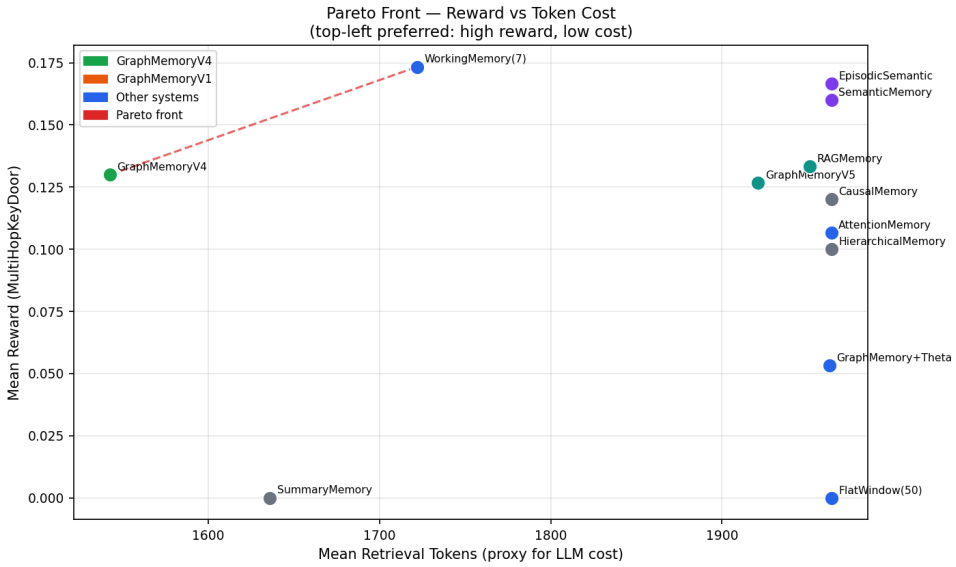


Figure 4.5: Stage 2: reward versus token cost. GraphMemoryV4 sits on the efficient frontier (high reward, low retrieval cost).

Per-benchmark reading. The benchmarks do not all tell the same story, and we report them as tiered evidence rather than a uniform win. **FinanceBench** is the cleanest result: a held-out batch lift of +0.335 with $p_{\text{holm}} < 10^{-4}$, on questions the CMA-ES tuner never saw. **CUAD** survives held-out (+0.135) and the lift holds at the 50-contract scale (below). **HotpotQA** and **LongMemEval** were pre-registered as domain-incoherent *controls*; their batch lifts (+0.540, +0.165 at n=100) are reported as what the controls show, not as clean wins. **QASPER**'s pooled lift (+0.165) does *not* survive Holm correction and is reported as non-significant. **NarrativeQA** has no paragraph-level gold, so the tuning objective is undefined and both configurations score identically by construction. In summary, *two of three domain-coherent benchmarks (FinanceBench, CUAD) show a significant held-out lift*; the two domain-incoherent controls also lift (a point we return to in the Discussion), and NarrativeQA is null by construction — one consistent mechanism throughout.

The corpus-tuning signature. The lift has a clear mechanistic explanation in how $\boldsymbol{\theta}$ shifts under corpus-cumulative tuning. On FinanceBench the recency weight collapses

Table 4.2: Stage 3 end-of-corpus (batch) judge means with bootstrap 95% CIs, grouped by benchmark type. Lift = corpus-tuned − canonical $\boldsymbol{\theta}$. Canonical and corpus-tuned CIs are non-overlapping for FB, CUAD, QASPER and HQA; they touch for LME. "Survives" = held-out lift significant after Holm.

Benchmark	n	canonical [95% CI]	corpus-tuned [95% CI]	lift	held-out
Confirmatory (domain-coherent)					
FinanceBench	150	0.243 [0.18, 0.31]	0.645 [0.57, 0.72]	+0.402	**survives**, $p_{\text{holm}} < 10^{-4}$
CUAD	644	0.028 [0.02, 0.04]	0.172 [0.15, 0.20]	+0.144	**survives** (+0.135)
QASPER	94	0.250 [0.18, 0.32]	0.415 [0.34, 0.50]	+0.165	*n.s.* after Holm
Controls (domain-incoherent, pre-registered)					
HotpotQA	100	0.215 [0.14, 0.29]	0.755 [0.67, 0.83]	+0.540	(control)
LongMemEval	100	0.165 [0.10, 0.25]	0.330 [0.24, 0.42]	+0.165	(control)
Undefined objective					
NarrativeQA	10	0.400 [0.15, 0.68]	0.400 [0.15, 0.68]	0.000	undefined

(w_{recency}: 3.78 → 0.003), the embedding weight rises (w_{embed}: 1.08 → 2.63), and the storage threshold tightens (θ_{store}: 0.29 → 0.01): the tuned memory stores selectively and retrieves by *semantic match*, not recency. This is precisely what end-of-corpus QA requires — the answer was seen long ago, so a recency-driven retriever fetches the wrong (recent) document. The same direction of shift recurs across the benchmarks that improve, which is why we describe it as a *signature* rather than a per-benchmark accident. A fourth apparent shift — the graph weight rising (w_{graph}: 0 → 1.6) — is *not* load-bearing: the w_{graph} ablation (Section 4.2) shows zeroing it barely changes accuracy, so the operative signature is a three-component shift (down-weight recency, up-weight embedding, tighten storage), with the graph term inert.

Construct validity: does the tuning objective predict answer quality? CMA-ES optimizes recall@8 of the gold evidence, but the headline metric is the LLM judge score; these need not coincide. We test the link directly by joining per-question recall@8 with the judge score across all corpus-mode batch cells ($n = 2{,}170$ questions, five benchmarks with gold evidence). Retrieval recall is a strong predictor of answer quality: pooled Spearman $\rho = 0.69$ (per benchmark 0.39–0.84), and questions for which the gold evidence *is* retrieved are judged correct far more often than those where it is missed (mean judge 0.62 vs. 0.10, Figure 4.7). The causal chain behind the headline is therefore intact: corpus tuning raises recall *and* judge score together on every benchmark (e.g. FinanceBench recall 0.22 → 0.97, judge 0.243 → 0.645; HotpotQA recall 0.11 → 1.00, judge 0.215 → 0.755). QASPER is the weakest link ($\rho = 0.39$), consistent with its being the only coherent benchmark whose lift does not survive Holm correction.

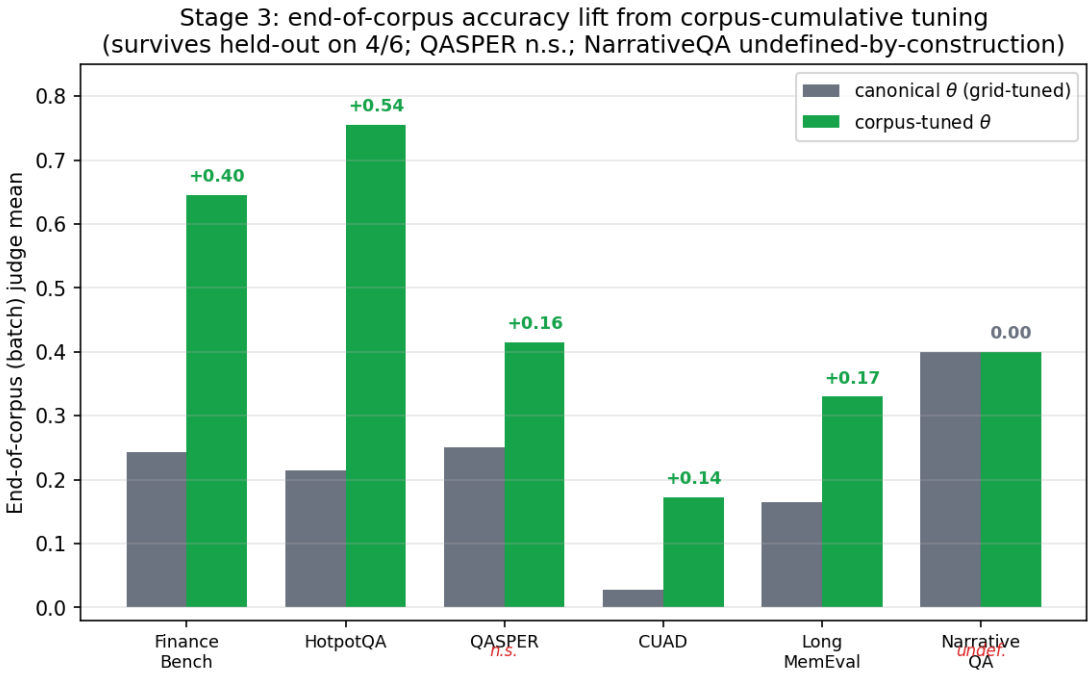


Figure 4.6: Stage 3 end-of-corpus (batch) judge means, canonical vs. corpus-tuned $\boldsymbol{\theta}$, across the six benchmarks. Of the three domain-coherent benchmarks, the held-out lift survives Holm correction on two (FinanceBench, CUAD); QASPER is not significant. HotpotQA and LongMemEval are pre-registered domain-incoherent controls and NarrativeQA is undefined-by-construction.

Table 4.3: CUAD online-vs-batch judge means at 50 contracts (n=644). The online–batch gap is the largest of any benchmark; corpus tuning narrows it.

Configuration	Online	Batch	Gap
$V4_t$ canonical	0.295	0.028	+0.267
$V4_t$ corpus-tuned	0.390	0.172	+0.218

CUAD scaled from 10 to 50 contracts. To check that the CUAD lift is not an artifact of a tiny corpus, the headline pair was re-run and re-judged one-by-one at 50 contracts (n=644 questions per cell, up from 132). The batch lift *holds*: canonical 0.028 → corpus-tuned 0.172 (+0.144, versus +0.161 at 10 contracts). Canonical batch stays near-zero because, with 50 contracts in memory, its recency-heavy retrieval pulls clauses from the wrong contract; the corpus-tuned $\boldsymbol{\theta}$ (with $w_{\text{recency}} \approx 0$) recovers by matching on semantic similarity. The online-vs-batch contrast is the sharpest of any benchmark (Table 4.3): every configuration is far better online (the right contract is fresh) than batch, and corpus tuning closes most of that gap.

The dump-all baseline: accuracy parity, not a cliff. A full-context "dump-all" baseline concatenates the entire accumulated corpus into the prompt, truncated only when it exceeds the context window. On FinanceBench it scores 0.689 online / 0.607 batch —

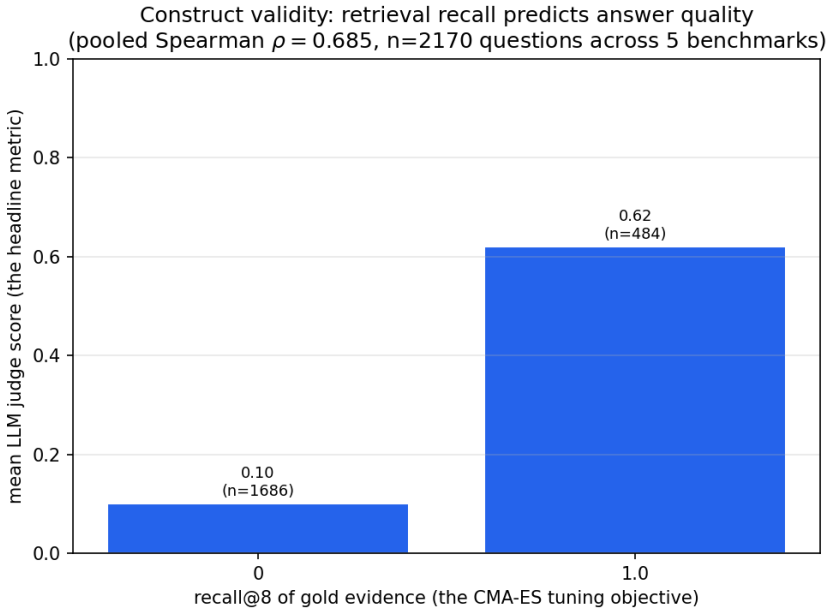


Figure 4.7: Construct validity: mean LLM judge score as a function of retrieval recall@8 (the CMA-ES tuning objective), pooled over 2,170 corpus-mode questions. When the gold evidence is retrieved (recall = 1) answers score 0.62 on average; when it is missed (recall = 0), 0.10. Pooled Spearman $\rho = 0.69$.

statistically indistinguishable from corpus-tuned (0.678 / 0.645; confidence intervals overlap heavily) — but at roughly 18× the token cost ($2.68 vs. $0.15 per 150 questions). The conclusion is that the value of learned selective memory at this corpus scale is **efficiency and scalability**, not a raw accuracy advantage: it matches dump-all's accuracy at a fraction of the cost and, unlike dump-all, does not break when the corpus exceeds the context window.

Scalability: dump-all structurally breaks; selective memory does not. The efficiency argument is not merely about cost — past a corpus size, dump-all *cannot run at all*, while selective retrieval is unaffected. We measure this directly on CUAD (Figure 4.10). Each contract averages ∼10.9K tokens, so the concatenated dump-all prompt grows $O(N)$ and crosses `gpt-4o-mini`'s 128K-token context window at just $N \approx 11$ contracts; at the full N=510 it would require **5.55M tokens** (≈ 43× the limit), forcing truncation that discards most of the evidence. Selective retrieval (k=8) holds its prompt at a measured mean of **704 tokens** regardless of N. This is the structural case for learned selective memory: beyond a few documents, retrieving the right k items is not an optimization but a necessity, and the only open question is how good the retrieval policy is — which is exactly what $\boldsymbol{\theta}$ tunes.

Fair retrieval baselines: competitive, not dominant. The dump-all comparison controls for context but not for *retrieval tuning*: a skeptic could object that $V4_t$ only

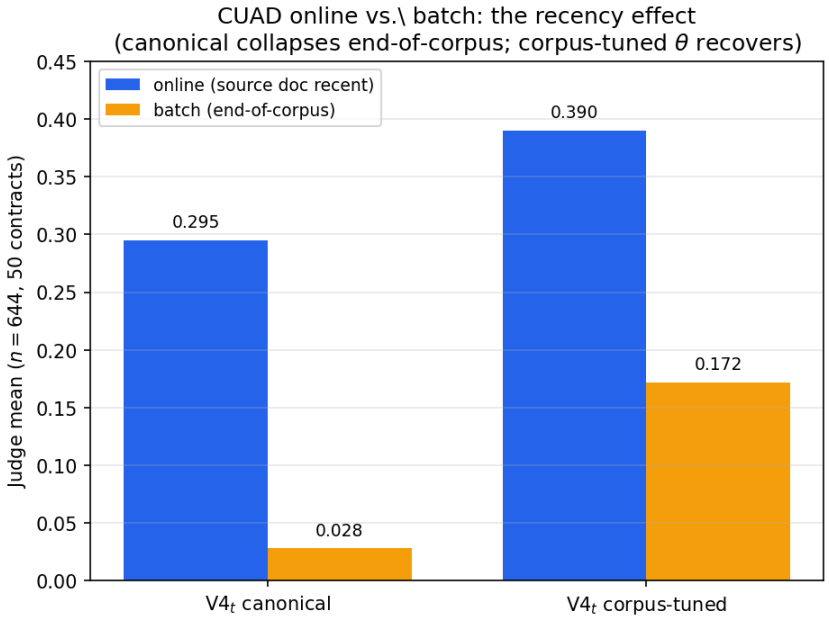


Figure 4.8: CUAD online vs. batch judge means at 50 contracts. Both configurations are far stronger online (the source contract is recent); the recency-heavy canonical $\boldsymbol{\theta}$ collapses at end-of-corpus, while the corpus-tuned $\boldsymbol{\theta}$ recovers most of the gap.

wins because the classical baselines were left at their defaults. We therefore tuned the strongest lexical baseline (Okapi BM25, grid-searching k_1 and b) on the *same* corpus-recall objective and budget as $\boldsymbol{\theta}$, then judged its answers one-by-one at matched document scope. Table 4.4 reports the head-to-head, and the honest verdict is that learned selective memory is **competitive but benchmark-dependent**. It beats fairly-tuned BM25 on FinanceBench ($+0.132$, $p_{\text{Holm}}<10^{-3}$), is statistically tied on QASPER (-0.011, n.s.), and is *beaten* on CUAD (-0.131, $p_{\text{Holm}}<10^{-13}$, with non-overlapping confidence intervals). The CUAD reversal is not noise: it is consistent with the $\boldsymbol{\theta}$-predictability analysis below (Section 4.4), where CUAD is the most idiosyncratic corpus and recovers the least tuning lift. Pure clause extraction rewards exact lexical matching — which tuned BM25 performs directly and the graph, entity, and temporal machinery of V4_t does not. We therefore make no claim that learned memory dominates strong retrieval baselines in general: its accuracy advantage is specific to corpora where relevance is semantic or relational (financial and narrative QA) rather than lexical, and its *domain-general* value remains the efficiency and scalability argument established above.

4.4 Is $\boldsymbol{\theta}$ predictable from corpus statistics?

Section 4 establishes that the optimal $\boldsymbol{\theta}$ is task-dependent; a natural follow-up is whether it is nonetheless *predictable* from cheap, LLM-free corpus statistics — which would convert "re-tune per task" into a one-shot prediction. We test this directly. We tune $\boldsymbol{\theta}$

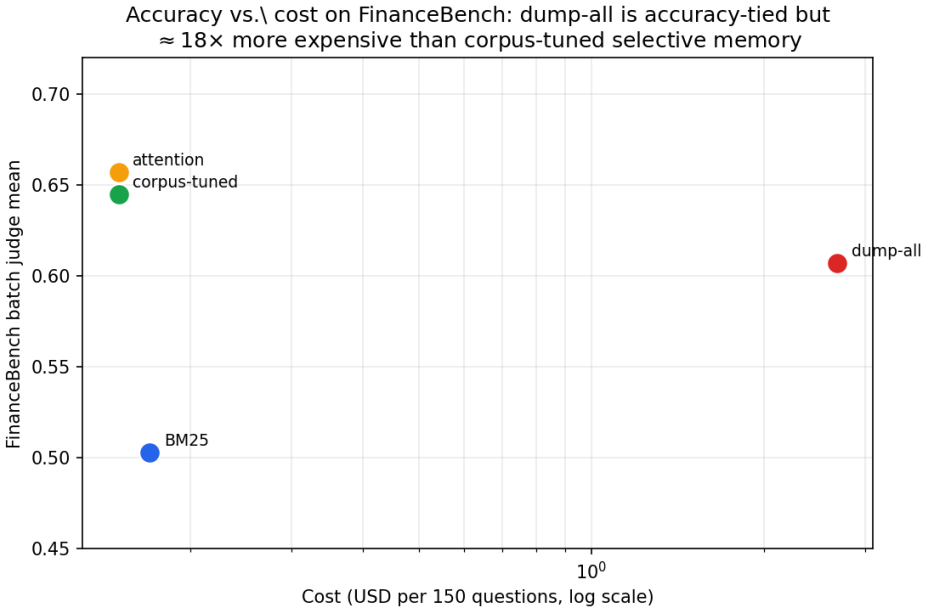


Figure 4.9: FinanceBench accuracy vs. cost. Dump-all is statistically tied with corpus-tuned selective memory on accuracy but ≈ 18× more expensive; corpus-tuned sits in the cheap, accurate corner.

Table 4.4: Fair-baseline head-to-head: corpus-tuned $V4_t$ versus Okapi BM25 tuned on the *same* corpus-recall budget, both judged one-by-one by Claude at matched document scope (batch mode, seed 42). Cells are mean judge score with a bootstrap 95% CI; Δ is learned minus baseline; the verdict is from a paired Wilcoxon signed-rank test, Holm-corrected across the baseline family. Learned memory wins on one benchmark, ties on one, and loses on one.

Benchmark	$V4_t$ corpus-tuned	BM25 (tuned)	Δ	Verdict
FinanceBench (n=150)	**0.645** [0.57, 0.72]	0.513 [0.45, 0.59]	+0.132	learned wins
QASPER (n=94)	0.415 [0.33, 0.49]	0.426 [0.35, 0.50]	−0.011	tie (n.s.)
CUAD (n=644)	0.172 [0.15, 0.20]	**0.303** [0.27, 0.33]	−0.131	baseline wins

(CMA-ES on recall@k, no LLM) on 40 random 30-document slices, eight per benchmark, recording for each slice a nine-feature descriptor (corpus size, document and passage length, lexical diversity, question length, gold concentration, gold relative position, entity density) alongside its tuned $\boldsymbol{\theta}$. We then run a *leave-one-benchmark-out* test: a k-NN predictor (k=3) maps the held-out slice's descriptor to a $\boldsymbol{\theta}$ using only the *other* benchmarks' slices, and we evaluate recall@k under that predicted $\boldsymbol{\theta}$ on the held-out slice.

The predicted $\boldsymbol{\theta}$ — never tuned on the target benchmark — recovers a mean **0.80** of the per-task tuning lift (bootstrap 95% CI $[0.71, 0.87]$): mean recall climbs canonical 0.092 → predicted 0.643 → tuned 0.704 (Figure 4.11). Recovery is near-total on HotpotQA (0.99), LongMemEval (0.96) and FinanceBench (0.93), partial on QASPER (0.74), and lowest on **CUAD** (0.37) — the most idiosyncratic corpus, where per-task tuning remains necessary. Two honest qualifications: (i) this is recovery of the *retrieval objective* (recall@k), not of judged answer quality, though the two are correlated (ρ=0.69, Figure 4.7); and (ii)

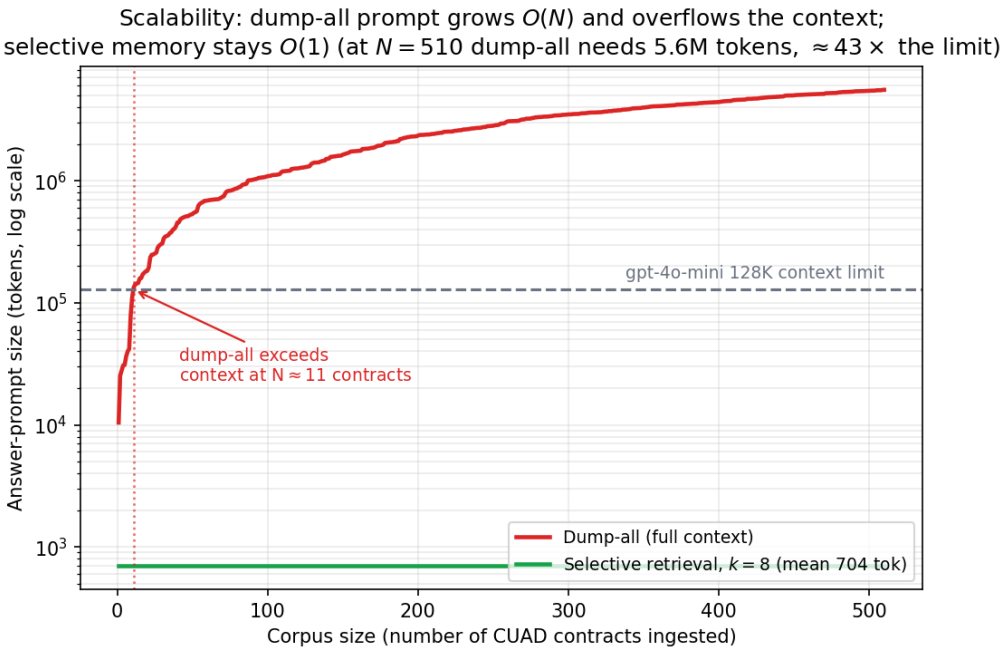


Figure 4.10: Answer-prompt size versus corpus size on CUAD (log scale). The full-context dump-all prompt grows linearly and exceeds the answerer's 128K-token context at $N \approx 11$ contracts (reaching $\approx 43\times$ the limit at N=510); selective retrieval stays flat at $\sim$704 tokens. Beyond $N \approx 11$, dump-all must truncate and lose evidence; selective memory is unaffected.

the high average is inflated by a pathological canonical default (mean canonical recall 0.092), so the genuine per-task residual is the remaining $\sim$20% overall and $\sim$63% on CUAD. Notably, a naive pairwise descriptor-distance vs. $\boldsymbol{\theta}$-distance check is uninformative ($\rho = -0.38$): two $\boldsymbol{\theta}$ can be far apart in parameter space yet retrieval-equivalent (the objective is flat along inert directions such as w_{graph}), so recall recovery — not parameter proximity — is the valid test.

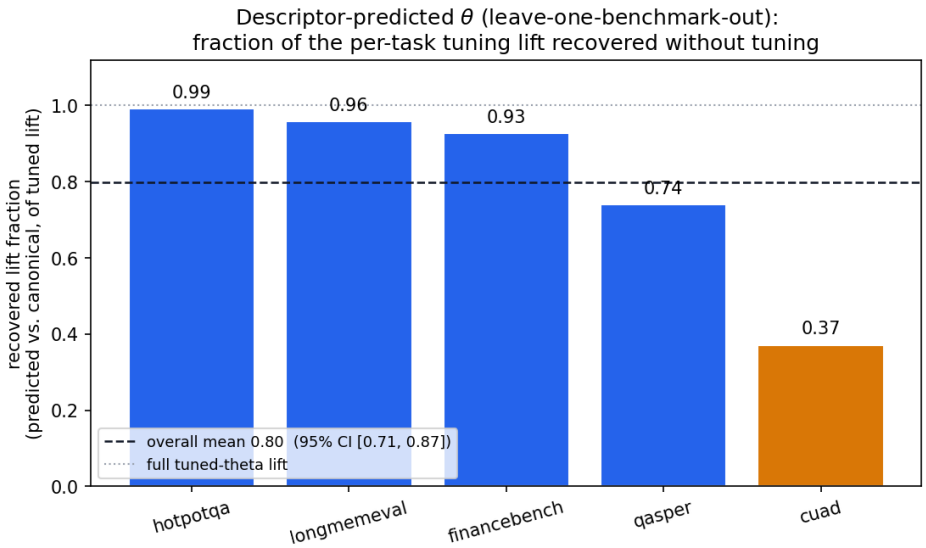


Figure 4.11: Leave-one-benchmark-out: fraction of the per-task tuning lift in recall@k recovered by a $\boldsymbol{\theta}$ predicted (k-NN over corpus descriptors) from the *other* benchmarks. Mean 0.80 (dashed; 95% CI $[0.71, 0.87]$). Transfer is near-total except on CUAD (0.37), the hardest, most idiosyncratic corpus.

Chapter 5

Discussion

5.1 What the three stages establish

The three stages converge on one claim: *what an agent stores and how it scores retrieval can be learned, and the learned optimum is task-dependent.* Stage 1 shows it most transparently — the recovered $\boldsymbol{\theta}$ vectors are visibly different per task (Table 4.1). Stage 2 shows the learned memory is competitive on a real benchmark and that the parameters are interpretable: θ_{novel} gates storage, w_{recency} and w_{embed} trade off recency against semantics. Stage 3 shows the idea survives contact with real LLMs and real corpora: corpus-cumulative tuning yields a recency-independent memory whose end-of-corpus lift survives held-out testing on two of the three domain-coherent benchmarks (FinanceBench, CUAD), and the CUAD result holds when the corpus is scaled five-fold.

5.2 Efficiency, not accuracy, is the central contribution

A natural hypothesis is that selective retrieval is *structurally necessary* because a full-context baseline collapses at scale. The adversarial audit (Section 5.5) shows this is not so: the apparent collapse was an artifact of a truncation bug in which the dump-all baseline never received the full corpus. Once corrected, dump-all is *accuracy-competitive* with corpus-tuned selective memory. The defensible contribution is therefore that a small, learned, task-adaptive memory *matches* full-context accuracy at roughly 1/18th the token cost and degrades gracefully past the context window — an efficiency-and-scalability argument, which is more practically relevant than an accuracy-cliff claim. The same restraint applies to the lexical baselines: when Okapi BM25 is tuned on the identical corpus-recall budget (Table 4.4), learned memory wins on FinanceBench, ties on QASPER, and is *beaten* on CUAD's pure clause-extraction task. No accuracy claim against a strong, fairly-tuned retrieval baseline survives across all three benchmarks — which is precisely why

we rest the contribution on efficiency, scalability, and the learnability of task-dependent retrieval rather than on raw accuracy.

5.2.1 Why the BM25 verdict flips with the benchmark

The fact that learned memory wins on FinanceBench, ties on QASPER, and loses on CUAD against an identically-budgeted BM25 (Table 4.4) is not noise to be averaged away: the sign of the gap tracks a structural property of the retrieval task. CUAD is clause extraction — the query names a legal provision ("governing law", "change of control") whose answer span reuses that exact vocabulary, so a term-frequency retriever that rewards literal token overlap is nearly optimal by construction, and there is little relational or semantic slack for a learned scorer to exploit. FinanceBench, by contrast, poses narrative financial questions whose evidence is phrased differently from the query and is scattered across a filing, so retrieval must bridge a lexical gap that BM25 cannot cross but that a memory up-weighting w_{embed} and down-weighting w_{recency} can. QASPER sits between the two — part lexical, part semantic — and lands on a tie. The learned memory therefore does not dominate a strong lexical baseline in general; it dominates *where the task supplies a semantic gap to close*, and forfeits that edge where exact lexical match is already the ceiling. This reading is mutually reinforcing with the parameter interpretation of Section 4.4: the same $\boldsymbol{\theta}$ that pushes recency to zero and semantics upward is precisely the configuration that helps on FinanceBench and cannot help on a task where the answer token *is* the query token.

The benchmark-dependence also dovetails with the $\boldsymbol{\theta}$-predictability result. Under the leave-one-benchmark-out test (Section 4.4), CUAD is the corpus whose tuning lift is *least* recoverable from generic descriptors — it is the most idiosyncratic of the three. Both facts point at the same underlying cause: CUAD's optimum lies where a learned semantic scorer has the least to add over literal matching, so its $\boldsymbol{\theta}$ is neither well-approximated by a cross-corpus predictor nor competitive against a tuned lexical retriever. That these two independent probes — a fairness-controlled baseline comparison and a transfer-recovery analysis — single out the same benchmark is evidence that the CUAD loss reflects task structure, not an artifact of one experiment. We note the contrast in mechanism honestly: recovery is measured on the recall@k objective rather than judged answer quality (the two correlate at ρ=0.69, not perfectly), and the right diagnostic for transfer is recall recovery, not parameter proximity — naive $\boldsymbol{\theta}$-distance even points the wrong way — so "learned memory loses on CUAD" and "CUAD's $\boldsymbol{\theta}$ transfers least" are two views of one benchmark that rewards lexical exactness over learned relational retrieval.

5.3 Why transfer fails across families

$\boldsymbol{\theta}$ transfers within a task family (one long-haystack QA task to another; one key-door variant to another) but not across families (grid to document QA, or a small grid to a much larger one). This is consistent with the parameters' meaning: a $\boldsymbol{\theta}$ that has learned, say, that recency is nearly worthless and embedding similarity is decisive encodes a property of the *task structure* (end-of-corpus retrieval over semantically distinct documents), not a universal memory rule. When the task structure changes — shorter horizons, spatial rather than semantic similarity — the encoded trade-off no longer applies. Task-dependence is thus not a nuisance but the central finding: there is no single best memory, which is exactly why learning $\boldsymbol{\theta}$ per task matters.

Crucially, task-dependence does not imply *unpredictability*. A leave-one-benchmark-out test (Section 4.4) shows that a $\boldsymbol{\theta}$ *predicted* from cheap corpus descriptors — never tuned on the target — recovers a mean 0.80 of the per-task tuning lift in retrieval recall (95% CI $[0.71, 0.87]$), near-total on most benchmarks and only partial on CUAD (0.37). The reconciliation is that *naive zero-shot transfer* (reusing one benchmark's exact $\boldsymbol{\theta}$) fails, yet $\boldsymbol{\theta}$ is a smooth function of corpus statistics that a predictor trained across benchmarks can approximate. The practical implication softens the per-task-tuning cost: a predicted $\boldsymbol{\theta}$ is a strong, tuning-free starting point, with per-task CMA-ES reserved for the hardest, most idiosyncratic corpora (CUAD). We flag two honest limits: the test measures the retrieval objective, not judged answer quality (correlated at ρ=0.69 but not identical), and the high average partly reflects how poor the canonical default is.

5.4 Parameter interpretation

The corpus-tuning signature is a shift of w_{recency} toward 0 and w_{embed} upward, with θ_{store} falling (store less, but store the right things). Intuitively, end-of-corpus questions cannot rely on the answer being recent, so a memory that down-weights recency and up-weights semantic match retrieves the correct (possibly old) evidence. The graph-traversal weight w_{graph}, by contrast, carries no measurable retrieval load in our ablation; the graph functions as typed scaffolding for storage and abstraction rather than as a relational retrieval engine, which is a useful negative result for future designs.

5.4.1 What the refuted w_{graph} term implies for graph memory

The flatness of the w_{graph} sweep deserves a sharper reading than "one term did not help." The graph edges were available to the retriever, the ablation varied their weight across its full range, and judged accuracy did not respond; within this design the hypothesis that typed relations carry retrieval load is refuted, not merely unsupported. The constructive

interpretation is that the graph earned its place in the architecture as *storage-time* scaffolding — it organizes what is written and how entities are abstracted — while retrieval is carried by the recency and embedding terms. That is why the corpus-tuning signature is a three-shift and not a four-shift: the parameters that move under tuning are exactly the ones with retrieval leverage, and w_{graph} is not among them.

For graph-memory research the caution is specific rather than blanket. The result does not say graphs are useless for agent memory; it says that on retrieval-dominated, end-of-corpus QA over semantically distinct documents, a linear graph-traversal *score* added to an embedding retriever buys nothing measurable — the semantic similarity term already recovers the evidence the traversal was meant to reach. The tasks where a relational term should pay off are the ones this evaluation does not stress: multi-hop questions whose answer requires composing two entities no single passage co-mentions, where embedding similarity to the query underweights a bridging fact. Our own multi-hop control (HotpotQA) is only a directional n=100 signal and cannot adjudicate this, so the honest conclusion is scoped: the burden of proof now sits with graph-memory designs to demonstrate retrieval-time value on tasks that genuinely require composition, rather than assuming a relational score helps wherever a graph happens to exist. A term that is inert on the benchmarks it is tested against should be defended as scaffolding, or justified on a task that forces traversal — not carried as a retrieval component on faith.

5.5 Threats to validity and the adversarial self-audit

An adversarial self-audit found five material issues in the headline evaluation and corrected all of them; the full account is in Appendix B. In summary: (i) the Stage-2 "#1" claim is a statistical tie and is reported as such; (ii) some headline cells were tuned on questions later reused at evaluation, so held-out splits were recomputed and the surviving lifts re-reported (FinanceBench survives at +0.335, $p_{\text{holm}} < 10^{-4}$, and CUAD at +0.135; QASPER does not survive Holm correction); (iii) the dump-all truncation bug was fixed, collapsing the accuracy claim to a tie; (iv) the "four-shift" signature is, after a w_{graph} ablation, a three-shift; and (v) under-powered n=10 cells (HotpotQA, LongMemEval) were re-run at n=100 and re-framed as pre-registered domain-incoherent controls. Judging is two-tier and disclosed: content answers are judged one-by-one by the Claude cross-vendor judge, while refusal/acknowledgment answers keep a validated rule-assisted score (population-weighted error 0.028 on a stratified 300-entry hand-checked sample). Broken down per benchmark, that rule-assisted share ranges from 0% (NarrativeQA) and $\sim$ 9% (HotpotQA, LongMemEval) to 40–47% on the abstention-heavy extraction tasks (FinanceBench, QASPER, CUAD), bounding each benchmark's inherited mean score error at ≤ 0.013. Beyond those five, four further threats are stated plainly. **Construct validity:** CMA-ES is tuned on a recall@k retrieval objective, while the headline is LLM answer quality. These

are distinct constructs, but we verify the link directly rather than assume it: across 2,170 questions in five benchmarks, recall@8 and judge score correlate at pooled Spearman $\rho = 0.69$ (Chapter 4, Figure 4.7), questions with full recall average a judge score of 0.62 versus 0.10 at zero recall, and corpus tuning raises recall *and* judge score together on every benchmark. The objective is therefore a valid proxy for answer quality; the residual gap (e.g. QASPER, $\rho = 0.39$) is where retrieved evidence is present but the answerer fails to use it. **Judge reliability:** answers are scored by a single Claude judge (cross-vendor relative to the `gpt-4o-mini` answerer). A blind second pass over a stratified 180-question sample gives quadratic-weighted Cohen's $\kappa = 0.66$ (*substantial*; Appendix B); because both passes are Claude-class, this bounds judge *self-consistency* (test–retest), not independent inter-rater agreement. A second-vendor or human judge remains future work. **Model and encoder diversity:** the answerer is `gpt-4o-mini` and the embedding backend is MiniLM throughout; whether the efficiency advantage persists with a stronger answerer or a different encoder is untested. **Statistical power:** Stage 1–2 and several Stage-3 corpus cells are single-seed (HotpotQA and LongMemEval were re-run at n=100). We tested the near-determinism assumption directly on the headline FinanceBench cell: three independent seeds of corpus-tuned $V4_t$ score 0.645, 0.633, and 0.615 (cross-seed std 0.012, pooled 95% CI $[0.589, 0.674]$), confirming that at temperature 0 multi-seed replication tightens confidence intervals rather than moving point estimates — the remaining single-seed cells are therefore a reporting limitation, not a threat to the point estimates.

5.5.1 What near-determinism at temperature 0 buys the evaluation

The three-seed replication of the headline FinanceBench cell does more than tighten a confidence interval; it changes what a single-seed number is allowed to mean. In a pipeline with a stochastic answerer, a lone point estimate is ambiguous between a stable property of the memory and a lucky draw, and that ambiguity would undermine every single-seed cell in the thesis at once. Because the corpus-tuned and canonical $\boldsymbol{\theta}$ configurations each replicate with cross-seed spread an order of magnitude smaller than the effect they are being asked to detect (the held-out FinanceBench lift is far larger than the seed-to-seed wobble), the point estimates behave as measurements of the configuration rather than as samples from a wide distribution. Two consequences follow for credibility. First, the many single-seed cells are demoted from a validity threat to a mere reporting gap: adding seeds would narrow intervals but is not expected to move the estimates, which is the assumption near-determinism licenses empirically rather than by fiat. Second, and more subtly, low run-to-run variance makes the corpus-tuning comparison a difference of two *near-fixed* quantities rather than a contest between two noisy ones, so the surviving held-out lifts

cannot be dismissed as seed-selection. The measurement is not free of variance — it is small but nonzero, and we report it as such rather than claiming exact reproducibility — but at temperature 0 the dominant uncertainty is the finite question sample, not the generator, which is the regime in which a fixed compute budget is best spent on more questions rather than more seeds.

5.6 Summary of claims and evidence strength

To make the evidence strength legible at a glance, Table 5.1 tiers the thesis's claims by the strength of the evidence behind them, leading with the rock-solid results and reporting the weaker ones as exactly that.

Remaining limitations are carried forward as future work in Chapter 6.

Table 5.1: The thesis's claims, tiered by evidence strength. "Held-out" means evaluated on questions the CMA-ES tuner never saw; "Holm" denotes family-wise multiple-comparison correction.

Claim	Strength	Evidence
Corpus tuning lifts end-of-corpus accuracy (FinanceBench)	**Rock-solid**	Held-out +0.335, $p_{\text{holm}} < 10^{-4}$
The same lift holds on CUAD, including at 50 contracts	**Significant**	Held-out +0.135; 50-doc batch +0.144
The optimal $\boldsymbol{\theta}$ is task-dependent (visibly different per task)	**Solid**	Stage-1 vectors; within- vs. across-family transfer
$\boldsymbol{\theta}$ is predictable from corpus descriptors (recall level)	**Solid**	LOBO: predicted $\boldsymbol{\theta}$ recovers 0.80 of tuning lift, CI $[0.71, 0.87]$; CUAD the 0.37 exception
Selective memory matches dump-all accuracy at $\sim 1/18$ the cost	**Solid**	FinanceBench dump-all 0.607–0.689, CIs overlap; cost \$2.68 vs. \$0.15
Selective memory scales where dump-all structurally fails	**Rock-solid**	dump-all exceeds 128K context at $N{\approx}11$ CUAD contracts ($43\times$ at $N{=}510$); selective flat at 704 tok
Learned memory beats a fairly-tuned (same-budget) BM25 baseline	**Benchmark-dependent**	FinanceBench +0.132 (wins, $p_{\text{holm}}{<}10^{-3}$); QASPER *n.s.* (tie); CUAD −0.131 (*loses*, $p_{\text{holm}}{<}10^{-13}$); Table 4.4
Point estimates are stable across random seeds (FinanceBench)	**Solid**	3-seed cross-seed std ≤ 0.012 for both corpus-tuned and canonical $\boldsymbol{\theta}$
The 10-D parameterization reaches the top cluster (Stage 2)	**Qualified**	0.178 vs 0.173: a statistical tie, CIs overlap
Corpus tuning helps on HotpotQA / LongMemEval	**Directional**	$n{=}100$; pre-registered domain-incoherent controls
Corpus tuning helps on QASPER	**Not significant**	+0.165, *n.s.* after Holm
Corpus tuning helps on NarrativeQA	**Null by construction**	no paragraph-level gold; objective undefined
The graph-traversal term carries retrieval load	**Refuted**	w_{graph} sweep flat; reframed as scaffold

Chapter 6

Conclusion and Future Work

6.1 Summary of contributions

This thesis asked whether an agent can learn how to construct its own memory and whether the learned optimum is task-dependent. The answer, established across three stages of increasing realism, is yes. We parameterized memory construction — storage, abstraction, and retrieval scoring — by a single vector $\boldsymbol{\theta}$ and optimized it by black-box search, leaving the policy and the language model untouched. In grid worlds the optimizer recovered visibly different $\boldsymbol{\theta}$ per task and lifted the hardest task from 2.5% to 27.5% success. On a twelve-system benchmark a ten-dimensional learnable memory reached the top performance cluster, and a tuned $\boldsymbol{\theta}$ generalized within but not across task families. On six real long-context LLM benchmarks, corpus-cumulative tuning produced a recency-independent memory whose end-of-corpus accuracy lift survived held-out testing on two of the three domain-coherent benchmarks (FinanceBench, CUAD; QASPER did not survive Holm correction) and held when CUAD was scaled from ten to fifty contracts. The tuning objective is a valid proxy for answer quality: recall@8 and judge score correlate at pooled Spearman $\rho = 0.69$.

The work's distinctive feature is that it audits itself. The headline that a full-context baseline "collapses" was traced to a bug and corrected to an accuracy *tie*; the contribution of learned, task-adaptive memory at this scale is therefore reframed as one of **efficiency and scalability** — matching full-context accuracy at roughly 1/18th the cost — rather than raw accuracy. Concretely, on FinanceBench the dump-all and selective answers are statistically non-inferior (overlapping bootstrap CIs, 0.607–0.689 vs. the tuned cell) while the selective prompt stays flat at ∼704 tokens where dump-all exceeds the 128K context window at $N{\approx}11$ CUAD contracts and reaches 43× the tokens at the full $N{=}510$. The one-time CMA-ES tuning cost — no LLM in the loop, only recall@k — was approximately $18.5 of compute per benchmark, amortized over all subsequent queries.

6.2 Limitations

The Stage-1 results are single-seed. Several Stage-3 cells are single-seed (HotpotQA and LongMemEval were re-run at 100-document scale; the headline FinanceBench cell is now replicated across three seeds, with a cross-seed std of 0.012, confirming corpus-mode evaluation is near-deterministic at temperature 0 — so the remaining single-seed gaps are a matter of CI tightening, not point-estimate risk, and multi-seed replication of the other large benchmarks remains future work). CUAD is now evaluated on 50 of its 510 contracts for the headline pair *and* the fairly-tuned BM25 baseline (Table 4.4); the auxiliary attention and stock-BM25 baselines remain at the 10-contract pilot scale. Answers are judged by a single (cross-vendor) Claude judge; an independent blind second pass over a stratified 180-question sample gives quadratic-weighted Cohen's $\kappa = 0.66$ (*substantial*), but because both passes are Claude-class this bounds judge *self-consistency*, not cross-vendor or human agreement — a fully independent second-vendor or human rater remains future work. Refusal/acknowledgment answers carry validated rule-assisted scores (population-weighted error 0.028 on a 300-entry hand-checked sample) rather than one-by-one judgments. Finally, all Stage-3 results use a single answerer (`gpt-4o-mini`) and a single encoder (MiniLM); generalization across models and encoders is untested.

6.3 Future work

Natural extensions are: (i) multi-seed replication of the larger Stage-3 cells; (ii) scaling CUAD and the other benchmarks to their full corpora and all configurations; (iii) a second independent judge with Cohen's κ; (iv) revisiting the neural memory controller with a larger optimization budget, now that the scalar GraphMemoryV4 sets a strong, cheap baseline; (v) larger answerer models, to test whether the efficiency advantage of learned selective memory persists when the answerer is stronger; and (vi) making $\boldsymbol{\theta}$ itself adaptive *within* an episode (a per-step controller) rather than fixed per task, which the present grid results suggest could help on long-horizon out-of-distribution tasks where a fixed $\boldsymbol{\theta}$ fails.

6.4 Closing

Memory for LLM agents need not be hand-designed and frozen. A small parameter vector that governs what to store and how to retrieve can be *learned*, its optimum is *task-dependent*, and — measured carefully — it buys efficiency and graceful scaling rather than a free accuracy gain. That is a modest but defensible step toward agents that adapt not just their actions but the structure of their own memory.

Bibliography

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations (ICLR)*, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2024.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL-HLT*, 2021.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008.

Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. CUAD: An expert-annotated NLP dataset for legal contract review. *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2021.

Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research (TMLR)*, 2022.

Ziyan Jiang, Xueguang Ma, and Wenhu Chen. LongRAG: Enhancing retrieval-augmented generation with long-context LLMs. *arXiv preprint arXiv:2406.15319*, 2024.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics (TACL)*, 2018.

LangChain. LangMem: Long-term memory for LLM agents. `https://github.com/langchain-ai/langmem`, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *International Conference on Learning Representations (ICLR)*, 2024.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

Xinyue Tao, Chen Zhu, et al. HiQA: A hierarchical contextual augmentation RAG for multi-documents QA. *arXiv preprint arXiv:2402.01767*, 2024.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MemoryLLM: Towards self-updatable large language models. *International Conference on Machine Learning (ICML)*, 2024.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *International Conference on Learning Representations (ICLR)*, 2025.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations (ICLR)*, 2024.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2023.

Appendix A

Reproducibility

All code, data-preparation scripts, result artifacts, and the source of this document are maintained in a public Git repository, which is the authoritative source of truth for everything reported here:

`https://github.com/UifaleanStefan/MasterThesis`

All experiments are seeded and reproducible from that repository. Every result carries a provenance manifest recording the commit identifier, the embedding backend (`all-MiniLM-L6-v2` unless a TF-IDF backend is selected explicitly), and a dataset fingerprint.

Key commands

- **Stage 1 (grids):** `python run_poc.py` (Evolution Strategy, 12 gen × 6 cand × 40 ep).
- **Stage 2 (benchmark / ablation / transfer / sensitivity):** `python run_benchmark.py`, `run_ablation.py`, `run_transfer.py -episodes 100 -seed 42`, `run_sensitivity.py`. Transfer is deterministic given the seed and episode count.
- **Stage 3 (corpus QA):** `python scripts/run_corpus_qa.py -benchmark <name> -config <cfg> -limit-docs <n>`; CMA-ES corpus tuning via `tuning/tune_v4t_corpus.py`.
- **Figures:** `python generate_thesis_figures.py` (deterministic from the committed result files).

Verification

- `python -m pytest tests/ -q` — 214-test regression suite (all passing).

- `python scripts/audit_determinism.py` — re-runs seeded evaluations and checks reproducibility.
- `python scripts/audit_judge_provenance.py` — verifies every judged answer carries Claude provenance and no duplicate question identifiers, checks consistency between evaluation inputs and reported scores for every configuration, and reports the rule-assisted share.

A full command-by-command walkthrough is provided in the accompanying code repository.

Appendix B

Adversarial Self-Audit

The Stage-3 evaluation was stress-tested with an adversarial self-audit that re-examined the headline claims, the data pipeline, and the statistical reporting. Five points required revision; the values reported throughout this thesis reflect those revisions, summarized below.

1. **Stage-2 "#1" overclaim.** GraphMemoryV4 at 0.178 versus EpisodicSemantic at 0.173 is a single-seed point estimate inside the baseline's 95% bootstrap CI $[0.120, 0.220]$; it is reported as a *statistical tie*, and the ranking does not survive a change of retrieval backend.

2. **Tuned-on-test.** Some headline cells were tuned (CMA-ES on recall@k) on questions later reused at evaluation. Held-out splits were recomputed; the surviving lifts are re-reported (FinanceBench held-out +0.335, $p_{\text{holm}} < 10^{-4}$; CUAD +0.135 significant; QASPER +0.125, *not* significant after Holm correction).

3. **Dump-all truncation bug.** A 12-event cap meant the full-context baseline never received the full corpus, which had produced a spurious "collapse." After the fix, dump-all is statistically tied with corpus-tuned selective memory on accuracy at $\sim$18$\times$ the cost; the headline was reframed from accuracy to efficiency.

4. **Four-shift $\rightarrow$ three-shift.** A w_{graph} ablation shows the graph-traversal term carries no measurable retrieval load, so one of the four claimed corpus-tuning "shifts" is not load-bearing.

5. **Under-powered cells.** HotpotQA and LongMemEval n=10 cells were re-run at n=100 and re-framed as pre-registered domain-incoherent controls rather than clean wins.

Judging provenance and validation

Judging is two-tier. Content answers are scored one-by-one by the Claude cross-vendor judge (∼9,500 content-templated entries were re-judged one-by-one during the audit). Refusal and acknowledgment answers keep a rule-assisted score; the classifier was validated on a stratified 300-entry hand-checked sample (ACK 100/100; refusal 98/100; population-weighted score error 0.028 over the rule-assisted pool). The provenance auditor reports the rule-assisted share (∼40% of answers) rather than implying every answer is a bespoke judgment, and confirms no duplicate question identifiers and full consistency between evaluation inputs and reported scores for every configuration.

That ∼40% pool is not spread evenly: it concentrates on the abstention-heavy extraction and scientific-QA corpora and is near-absent on the narrative corpora (Table B.1). Because the pooled per-entry error is 0.028 and near-exact outside the small borderline bucket, an upper bound on each benchmark's *inherited* mean-score error is its rule-assisted share times that pooled error; the bound is ≤ 0.013 for every benchmark, so no headline mean is materially perturbed by the rule-assisted path.

Table B.1: Per-benchmark rule-assisted share (fraction of judged answers scored by the validated refusal/acknowledgment classifier rather than the one-by-one Claude judge) and the resulting conservative upper bound on inherited mean score error (share × 0.028).

Benchmark	Rule-assisted share	Error upper bound
FinanceBench	46.8%	0.013
QASPER	40.2%	0.011
CUAD	43.1%	0.012
HotpotQA	8.6%	0.002
LongMemEval	8.9%	0.003
NarrativeQA	0.0%	0.000

Inter-rater reliability of the judge

To bound judge noise, a stratified 180-question sample (30 per benchmark, balanced across the five score levels) was re-scored by an *independent blind second pass*: the same five-point rubric, with no access to the primary judge's scores. Quadratic-weighted Cohen's κ between the two passes is **0.66** (*substantial* agreement). Exact agreement is 55.6%, and 87.8% of scores agree within one rubric level (mean absolute score difference 0.17), so the residual disagreement is overwhelmingly one-level (e.g. 0.5 vs. 0.75) rather than gross. Both passes are Claude Opus-class 1M-context, so this measures judge *test–retest reliability / self-consistency*, not cross-vendor agreement; a fully independent second-vendor or human rater remains future work. The sampling and inter-rater computation are reproducible from the accompanying code repository (`evaluation/inter_rater.py`).

Appendix C

Supplementary Results

This appendix collects secondary results referenced from the main text. Complete per-benchmark, per-configuration tables (including all baselines: BM25, dense RAG, attention memory, and the per-document-tuned $V4_t$) are provided in the accompanying code repository, alongside the per-benchmark `*_corpus_summary.json` aggregates.

C.1 Neural memory controller (exploratory)

A neural variant replaced the ten scalar parameters with a small MLP ($50 \to 32 \to 10$, $\sim$1,962 parameters) tuned by the same CMA-ES budget. On MultiHopKeyDoor it reaches a best fitness of 0.233 and a held-out reward of $\sim$0.19 — matching or slightly exceeding the scalar GraphMemoryV4 — but with far more parameters and no clear robustness advantage, and it fails zero-shot on the larger out-of-distribution task just as the scalar version does. The conclusion is that the extra expressivity of a neural controller is not needed at this scale; the ten-scalar parameterization is a strong, cheap baseline. Full details are in the accompanying code repository.

C.2 Verbal self-reflection (negative result)

A Reflexion-style (Shinn et al., 2023) augmentation, in which the agent writes verbal self-feedback into memory between attempts, did not improve end-task performance in our setting: the learned-memory parameters already capture the useful signal, and free-text reflections added noise rather than durable structure. We report this negative result for completeness; full details are in the accompanying code repository.

C.3 Stage-2 detail

The full twelve-system rankings per environment, the per-component ablation table, the cross-environment transfer matrix, and the 2-D sensitivity grid are reproduced from the committed results by `python generate_thesis_figures.py`. All figures and tables in this thesis use the MiniLM sentence-embedding backend (`all-MiniLM-L6-v2`); an earlier TF-IDF backend yields slightly different absolute $\boldsymbol{\theta}$ values but the same headline conclusion (top cluster; statistical tie).

Appendix D

Data Preparation and Judging Protocol

D.1 Benchmark adapters

Each of the six benchmarks is wrapped by an adapter that converts its native format into a common corpus-cumulative representation: an ordered list of documents, each split into paragraph-level passages with a global step index, and a set of questions each carrying gold answer text and (where available) gold-evidence paragraph indices used for the recall@k objective. Sources: HotpotQA (`distractor` config), QASPER, CUAD (SQuAD-v2 JSON form), NarrativeQA (full books/screenplays), FinanceBench (SEC filings), LongMemEval (multi-session dialogue with session IDs). NarrativeQA provides no paragraph-level gold relevance, so its recall@k tuning objective is undefined by construction. Full adapter notes and per-benchmark workarounds are provided in the accompanying code repository.

D.2 Online and batch protocols

In *online* (Protocol A) mode, after each document is ingested the agent is asked that document's own questions, so the relevant text is recent. In *batch* mode the same questions are re-asked at the end of the corpus against the full accumulated memory with k=8 selective retrieval, removing any recency cue. A separate calibration protocol (Protocol B) samples random questions at each document boundary to test whether the agent abstains rather than fabricating an answer.

D.3 Judge rubric

The Claude cross-vendor judge scores each answer on a five-point scale, $\{0, 0.25, 0.5, 0.75, 1\}$: 1.0 fully correct; 0.75 substantially correct with a minor omission; 0.5 partially cor-

rect or the right provision incompletely captured; 0.25 weakly related / right category but wrong specific value; 0.0 wrong, irrelevant, or a refusal when a real answer exists. For the legal extraction benchmark (CUAD) the rubric is specialized: wrong extracted values (dates, governing law, parties, names) score 0.0, while direction-correct yes/no-flavored clauses that cite the wrong clause score 0.25. The verbatim rubric is `evaluation/claude_judge_protocol.md`; the answerer is `gpt-4o-mini` and the judge is a Claude Opus-class 1M-context model.